ESTUDO COMPARATIVO DE INFRAESTRUTURA EM NUVEM

Plataforma Unooria

Análise técnica, operacional e financeira entre **GCP**, **AWS** e **Azure** para a Plataforma, as Experiências Imersivas, a API e o Uso de Inteligência Artificial

Escopo do estudo	Comparação exaustiva da pilha de nuvem por contrato B2G (10.000 estudantes, picos de 2.000 a 4.000 simultâneos às 8h da manhã, 100+ escolas) nos quatro eixos do produto: **Plataforma** (`apps/hub`, borda, banco, mensageria, identidade e observabilidade), **Experiências** (`apps/experiencias/*`, bundles estáticos, CDN e mídia), **API** (`apps/api`, computação *serverless*, jobs em segundo plano, SSE e WebSockets) e **Uso de IA** (`src/ai`, `apps/agente-professor`, busca semântica/estruturada e voz ao vivo).
Nuvem de referência	Google Cloud Platform (**GCP**), provisionada via Terraform no repositório de fundação (`1 contrato = 1 stack isolada`).
Nuvens comparadas	Amazon Web Services (**AWS**) e Microsoft Azure (**Azure**).
Restrição de operação	Plataforma desenvolvida e operada por **uma pessoa** (*simplicidade* > *sofisticação*), com meta de produção em novembro de 2026.
Data de referência	13 de outubro de 2026

Documento complementar à Documentação Técnica da Plataforma Unooria (`documentacao-latex/main.pdf`, Capítulos 6, 11 e 17) e ao Mapa de Produto (`PU-PRD-001`).

Sumário

1 Como ler este comparativo e premissas da plataforma 1

1.1 Estrutura do documento e convenções visuais . . . 1

1.1.1 Selos de aderência arquitetural . . . 2

1.2 Premissas de negócio e arquitetura que a nuvem precisa respeitar . . . 3

1.2.1 Requisitos técnicos determinantes por eixo . . . 3

2 Visão geral e matriz de equivalência 6

2.1 Topologia comparada nas três nuvens . . . 7

2.2 Matriz completa de equivalência serviço a serviço . . . 7

3 Eixo 1 — Plataforma (Borda, Hub, Banco, Mensageria e Operação) 9

3.1 Borda única, WAF e gestão de certificados TLS . . . 9

3.2 Hospedagem do Hub (`apps/hub`) e Identidade (OIDC) . . . 11

3.3 Banco de Dados Relacional, RLS, Rede Privada e Acesso Dev . . . 12

3.4 Mensageria de Telemetria Analítica (Push Autenticado) . . . 13

3.5 Identidade Federada (CI/CD), Segredos e Observabilidade . . . 14

4 Eixo 2 — Experiências (Bundles Estáticos, CDN e Armazenamento) 16

4.1 Bundles estáticos imutáveis, CDN e isolamento de origem (`-exp`) . . . 16

4.2 Mídia pesada e imagens geradas por IA (`-assets`) . . . 18

4.3 Guarda versionada de planos de aula e sourcemaps privados . . . 18

5 Eixo 3 — API (`apps/api`) 20

5.1 Por que a API não cabe em FaaS (Lambda / Functions) . . . 20

5.2 Peças Arquiteturais do Eixo API . . . 22

5.3 Síntese Comparativa do Eixo API 26

6 Eixo 4 — Uso de IA (Modelos, Agente, RAG e Voz Ao Vivo) 27

6.1 Topologia Comparada da Stack de IA 28

6.2 Peças Arquiteturais do Eixo Uso de IA 29

6.3 Síntese Comparativa do Eixo Uso de IA 36

7 Custos (TCO), Operação Solo e Matriz de Migração 37

7.1 Estimativa Comparada de TCO Mensal por Contrato (Produção) 37

7.1.1 Três lições econômicas da tabela de TCO 39

7.2 Operação por Uma Única Pessoa (Carga Cognitiva) 40

7.3 Matriz de Portabilidade e Esforço de Migração (Lock-in) 41

7.4 Conclusão e Diretriz Arquitetural 42

A Código-Fonte Mermaid dos Diagramas 44

A.1 Topologia Geral Comparada (Figura 2.1) 44

A.2 Topologia Comparada da Stack de IA (Figura 6.1) 45

1

Como ler este comparativo e premissas da plataforma

Este documento analisa, peça por peça, a infraestrutura em nuvem da **Plataforma Unooria** e compara a arquitetura atualmente implantada no **Google Cloud Platform (GCP)** com as suas traduções arquiteturais diretas em **Amazon Web Services (AWS)** e **Microsoft Azure**.

Diferente de um comparativo genérico de mercado, este estudo parte das **invariantes reais do código e do modelo de negócio** da Unooria — documentadas nas ADRs do monorepo e nos módulos Terraform do repositório de fundação. O objetivo é responder a três perguntas práticas:

1. **Equivalência técnica:** Qual serviço de cada nuvem cumpre exatamente o papel das peças atuais da Plataforma, das Experiências, da API e do Uso de IA sem violar as regras invioláveis de segurança, *tenancy* e tempo de resposta?
2. **Custo de operação para uma pessoa:** Onde a troca de nuvem reduz ou multiplica a carga cognitiva e operacional de um único desenvolvedor mantendo múltiplos contratos B2G?
3. **Estrutura de custos (TCO) e acoplamento:** Como se comportam o piso fixo mensal por contrato e o custo variável de IA nas três nuvens, e quais componentes têm portabilidade imediata versus acoplamento profundo?

1.1 Estrutura do documento e convenções visuais

O estudo está organizado nos quatro eixos solicitados, precedidos por uma visão executiva de equivalência e seguidos pela análise financeira e de migração:

- **Capítulo 2 — Visão geral e matriz de equivalência:** diagrama comparativo da topologia completa e tabela sintética de mapeamento serviço a serviço (`GCP` × `AWS` × `Azure`).
- **Capítulo 3 — Eixo 1: Plataforma:** borda única (Load Balancer, TLS e WAF), hospedagem do `apps/hub` (Next.js App Router), banco de dados PostgreSQL com *Row-Level Security* (RLS),

mensageria de telemetria (*push* autenticado), identidade federada (*keyless*) e observabilidade (logs, métricas, alertas e OTLP).

- **Capítulo 4 — Eixo 2: Experiências:** entrega de *bundles* estáticos imutáveis por versão (`apps/experiencias`) estratégia de *cache* e *rollback* em segundos via CDN, isolamento de origem para `postMessage`, mídia pesada (`-assets`), guarda versionada de planos de aula (`-planos`) e *sourcemaps* privados.
- **Capítulo 5 — Eixo 3: API:** runtime *serverless* do `apps/api` (NestJS), alocação contínua de CPU para *workers* em segundo plano pós-`HTTP 202`, concorrência por instância, *timeouts* longos para *streams* SSE e WebSockets de voz ao vivo, saída privada de rede (VPC) e execução de *migrations* via *job* isolado.
- **Capítulo 6 — Eixo 4: Uso de Inteligência Artificial:** a camada única (`AiProvider` com texto curto, saída longa de 64k *tokens*, JSON validado, geração de imagem com proporção nativa e TTS), o Agente de IA do Professor (`apps/agente-professor` em Python/ADK com sessões gerenciadas e dupla busca híbrida estruturada/layout com `pageSpan`) e a Ajuda por voz bidirecional (`Gemini Live`) e vídeo (`LiveAvatar`).
- **Capítulo 7 — Custos, operação solo e matriz de acoplamento:** estimativa comparada do piso fixo mensal por contrato e consumo variável de IA, impacto na rotina de operação de uma única pessoa e esforço real de portabilidade de cada peça.

1.1.1 Selos de aderência arquitetural

Cada componente analisado nos Capítulos 3 a 6 traz uma ficha técnica e selos que classificam o grau de aderência de cada nuvem aos requisitos da Unooria:

Selo	Significado técnico neste estudo
NATIVO / DIRETO	O serviço da nuvem atende ao requisito de forma nativa e direta, com configuração declarativa em Terraform e sem exigir peças intermediárias nem mudanças no código da aplicação.
EQUIVALENTE	A nuvem oferece capacidade funcional e de segurança equivalente, mas exige a combinação de dois recursos (ex.: fila + função consumidora, ou proxy de conexões separado) ou pequenos ajustes em variáveis/adaptadores.
EXIGE ADAPTAÇÃO	A nuvem atende ao caso de uso, porém impõe mudança relevante de arquitetura, reescrita de cliente/SDK (ex.: refatorar o agente ADK ou o pipeline de *chunking* de PDFs com páginas) ou salto expressivo no piso fixo de custo.
INCOMPATÍVEL	O serviço candidato viola uma premissa técnica da plataforma (ex.: limite rígido de 15 minutos ou congelamento de CPU no AWS Lambda para conexões WebSocket de 60 minutos e *jobs in-process* pós-202).

1.2 Premissas de negócio e arquitetura que a nuvem precisa respeitar

Qualquer comparação honesta entre GCP, AWS e Azure precisa julgar os serviços contra as restrições reais do produto. Uma solução que exige um time de SRE dedicado ou que encarece o piso ocioso de cada contrato em milhares de dólares é inviável para o modelo da Unooria, ainda que seja tecnicamente elegante.

Invariantes de negócio e operação da Plataforma Unooria

Modelo B2G	**1 contrato = 1 stack isolada de infraestrutura**, provisionada por Terraform em projeto/conta própria. Dentro do contrato, 100+ escolas compartilham a stack com isolamento lógico estrito por `escola_id` (*Row-Level Security* no PostgreSQL).
Escala por contrato	Cerca de **10.000 estudantes** ativos, com picos concentrados de **2.000 a 4.000 usuários simultâneos** no início do turno escolar (8h da manhã) e vales profundos à noite e nos fins de semana.
Equipe de operação	**Uma pessoa** desenvolve o código, mantém os módulos Terraform, responde a incidentes e publica versões. Todo serviço que exige manutenção de cluster (como Kubernetes/EKS/AKS/GKE) ou ajustes manuais de rede/certificados é descartado em favor de serviços gerenciados *serverless*.
Zero credenciais longas	Nenhuma chave estática de API ou JSON de conta de serviço em código, `.env` de produção ou segredos do GitHub. O CI/CD autentica por federação OIDC (*Workload Identity*) e os serviços em execução usam a identidade nativa do ambiente (ADC / Task Role / Managed Identity).

1.2.1 Requisitos técnicos determinantes por eixo

A Tabela 1.2 sintetiza os requisitos técnicos que funcionam como “filtro eliminatório” ao escolher o serviço correspondente em AWS e Azure.

Tabela 1.2 – Requisitos técnicos da Unooria que determinam a escolha dos serviços em nuvem

Eixo	Requisito crítico	Por que elimina soluções comuns de mercado
Plataforma	Borda única com roteamento por *hostname*, WAF OWASP e certificados validados por DNS sem janela sem TLS	O Hub, a API, as Experiências e os Assets respondem sob domínios próprios no mesmo balanceador. Adicionar um domínio nunca pode recriar o certificado de quem já está no ar (**ADR-57**, `loadbalancer.tf`).
Plataforma	PostgreSQL 16 privado com `FORCE ROW LEVEL SECURITY`, particionamento mensal e duas *roles* sem `BYPASSRLS`	Descarta bancos NoSQL e exige PostgreSQL gerenciado real, sem IP público, com acesso administrativo exclusivamente via túnel autenticado por IAM (IAP/SSM) e *connection pooling* para absorver escalonamento de contêineres.
Plataforma	Fila de telemetria com entrega *push* HTTP autenticada por OIDC e DLQ	A API centraliza a ingestão em `POST /telemetry/ingest`. A nuvem deve empurrar a mensagem autenticada para a própria API sem exigir um serviço *worker* separado rodando 24/7 só para fazer *polling* de fila.
Experiências	*Bundles* estáticos imutáveis em `/{expId}/{versao}/` com origem própria no CDN	Experiências nunca rodam em contêiner (*regra inviolável*). Precisam de domínio dedicado (`exp_domain`) para que o `targetOrigin` do `postMessage` do Hub não compartilhe origem com o restante da nuvem.
API	Contêiner *serverless* com alta concorrência (80 req/instância), CPU sempre alocada e *timeout* de até 3.600 s	A geração de planos e pacotes responde `HTTP 202` e segue processando em segundo plano no mesmo processo; o Agente do Professor mantém SSE por até 300 s; o mascote da Ajuda mantém WebSocket bidirecional por até 60 minutos.
Uso de IA	Saída longa (64k *tokens*), controle de orçamento de raciocínio, geração de imagem com *aspect ratio* nativo (`4:5`) e voz bidirecional	Planos de aula anuais cortam em 32k *tokens*; chamadas rápidas em sala de aula (`diagramar`) não podem estourar latência nem orçamento oculto de *thinking tokens* (**ADR-148**).
Uso de IA	Runtime gerenciado para agente Python (ADK) com sessões persistentes + busca estruturada (BNCC) e por *layout* de PDF com `pageSpan`	O Agente do Professor precisa listar 100% das habilidades de um ano por filtro exato e citar livro + intervalo exato de páginas (`p. X--Y`) sem manter banco vetorial provisionado por hora a custo proibitivo.

REGRA INVIOLÁVEL

Simplicidade operacional é requisito funcional. Quando uma nuvem oferece dois caminhos para o mesmo problema — por exemplo, rodar a API em Kubernetes (EKS/AKS) versus rodá-la em um serviço de contêineres *serverless* (ECS Fargate / Azure Container Apps) —, este estudo seleciona sempre o caminho **gerenciado e sem cluster**, preservando a regra de negócio de que toda a plataforma é operada por uma única pessoa.

2

Visão geral e matriz de equivalência

Antes de mergulhar nas particularidades de cada um dos quatro eixos, este capítulo apresenta o mapa completo de correspondência arquitetural entre a pilha atual da Unooria no **GCP** e as suas arquiteturas equivalentes na **AWS** e no **Azure**.

Síntese executiva da comparação entre as três nuvens	
GCP (Atual)	NATIVO / DIRETO É a nuvem onde o desenho atual atinge a **menor fricção operacional por contrato**: um único Load Balancer global agrega tanto contêineres (*Serverless NEGs* do Cloud Run) quanto buckets estáticos (*Backend Buckets* do GCS) sob o mesmo IP e WAF; o Cloud Run combina alta concorrência (80 req/instância), CPU sempre alocada, WebSockets de 60 minutos e *Jobs* de migração no mesmo modelo; o Pub/Sub faz *push* HTTP autenticado por OIDC direto para a API; e o ecossistema Vertex AI concentra Gemini (64k saída, imagem 4 : 5, Live voz), Agent Engine (ADK nativo com sessões) e Vertex AI Search (busca estruturada + *layout parser* com `pageSpan` cobrado por uso).
AWS	EXIGE ADAPTAÇÃO Oferece maturidade exemplar em rede, banco de dados (RDS Multi-AZ + RDS Proxy) e entrega estática (CloudFront + S3 com OAC), mas **fragmenta peças que no GCP são unificadas**: o ALB não aponta para buckets S3 (exigindo CloudFront na frente de tudo para unificar borda e WAF); o AWS Lambda é incompatível com os requisitos da API (limite de 15 min e modelo de execução), levando ao **ECS Fargate** (ou App Runner com ressalvas de VPC/Jobs); o SQS não faz *push* HTTP nativo sem peça intermediária (EventBridge Pipe / Lambda); e no eixo de IA, as *Bedrock Knowledge Bases* exigem um banco vetorial subjacente (como OpenSearch Serverless, com piso fixo elevado em OCUs, ou Aurora pgvector) e reescrita do agente ADK.
Azure	EQUIVALENTE É a alternativa **mais próxima do modelo operacional do Cloud Run** para computação e borda: o **Azure Front Door** roteia nativa-

mente para contêineres e Blob Storage com WAF integrado; o **Azure Container Apps** (ACA) entrega experiência muito similar ao Cloud Run (concorrência HTTP, *scale-to-zero* ou réplicas mínimas, sessões WebSocket longas e *Container Apps Jobs* para o `prisma migrate deploy`); e o PostgreSQL Flexible Server já traz *PgBouncer* embutido sem custo extra. No eixo de IA, combina Azure OpenAI (`GPT-4o` + `Realtime API`) e **Azure AI Search** com *Document Intelligence*, mas ainda exige portar o código do agente ADK e gerenciar índices cobrados por hora.

2.1 Topologia comparada nas três nuvens

A Figura 2.1 ilustra como os quatro eixos da Plataforma Unooria (Borda/Plataforma, Experiências, API/Dados e Uso de IA) se organizam em cada provedor de nuvem. O código Mermaid correspondente encontra-se no Apêndice A (Listagem A.1).

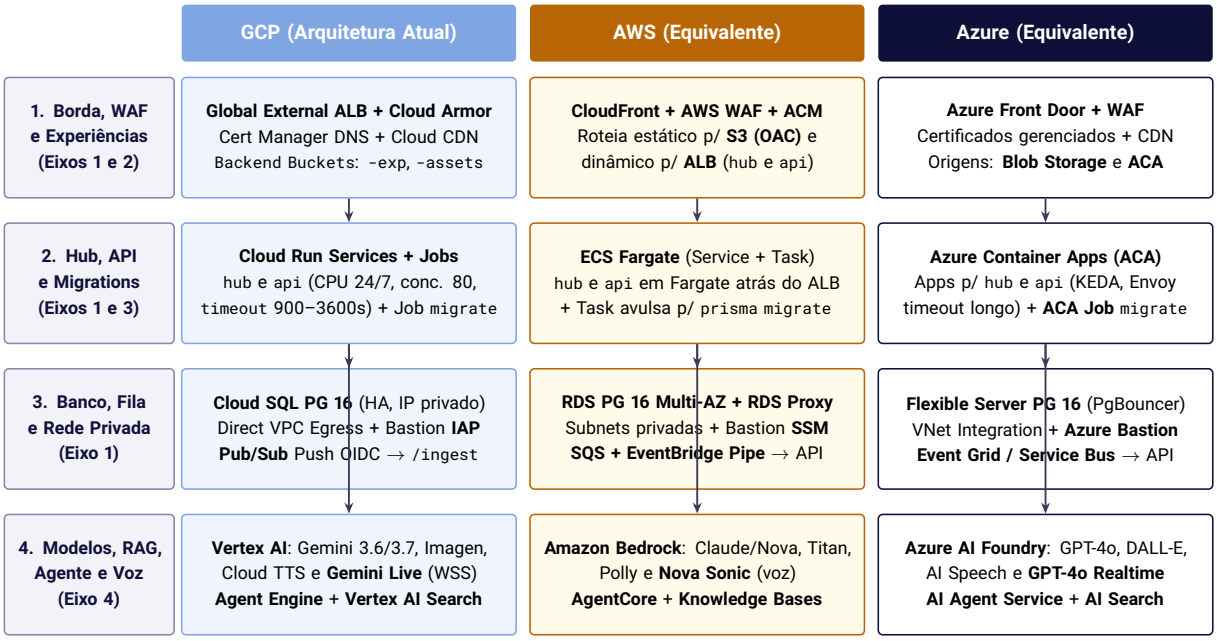


Figura 2.1 – Correspondência estrutural das quatro camadas entre GCP, AWS e Azure para um contrato da Plataforma Unooria.

2.2 Matriz completa de equivalência serviço a serviço

A Tabela 2.1 mapeia todos os recursos declarados nos arquivos `.tf` do repositório `Fundacao Plataforma Unooria` para os seus equivalentes diretos em AWS e Azure.

Tabela 2.1 – Mapeamento completo dos serviços da Plataforma Unooria entre GCP, AWS e Azure

Função na Unooria	GCP (Atual)	AWS	Azure
Eixo 1 — Plataforma (Borda, Hub, Banco, Rede, Mensageria, Segurança e Observabilidade)			
Isolamento por contrato	1 GCP Project por ambiente/contrato	1 AWS Account (via AWS Organizations)	1 Subscription ou Resource Group isolado
Entrada única HTTP/S	Global External Application Load Balancer	Amazon CloudFront + Application Load Balancer (ALB)	Azure Front Door (Standard/Premium)
WAF e *rate limit*	Cloud Armor Standard (OWASP CRS 3.3 + RPM/IP)	AWS WAF v2 (*Managed Rules* + *Rate-based rule*)	Azure WAF no Front Door (OWASP DRS + *Rate limit*)
Certificados TLS	Certificate Manager (validação DNS por host)	AWS Certificate Manager — ACM (validação DNS)	Front Door Managed Certificates / Key Vault
Hospedagem do `hub`	Cloud Run Service (`1--2 vCPU`, `min=1`)	ECS Fargate Service (ou AWS App Runner)	Azure Container Apps — ACA (`minReplicas=1`)
Banco relacional + RLS	Cloud SQL for PostgreSQL 16 (Regional HA, IP privado)	Amazon RDS for PostgreSQL 16 (Multi-AZ) + RDS Proxy	Azure Database for PostgreSQL Flexible Server (HA + PgBouncer)
Rede privada e saída	VPC + Direct VPC Egress + Cloud NAT + PSA	VPC (Subnets privadas) + NAT Gateway	Virtual Network (VNet) + VNet Integration + NAT Gateway
Acesso dev ao banco	VM Bastion (`e2-micro`) sem IP público + túnel IAP	Instância `t4g.nano` privada + SSM Session Manager	Azure Bastion (*Developer* / *Basic*) ou ACA túnel
Fila de telemetria (*push*)	Cloud Pub/Sub (tópico + *Push* OIDC + DLQ)	Amazon SQS + EventBridge Pipe (*API Destination*) + DLQ	Azure Event Grid (*Push* c/ Entra ID) ou Service Bus
Segredos de runtime	Secret Manager (injetado como *env* no boot)	AWS Secrets Manager / SSM Parameter Store	Azure Key Vault (referenciado no Container App)
Registro de imagens	Artifact Registry (limpeza em 7 dias, 20 tags)	Amazon ECR (*Lifecycle Policy* equivalente)	Azure Container Registry — ACR (retenção por política)
Federação CI/CD	Workload Identity Federation (OIDC GitHub)	IAM OIDC Identity Provider (`sts: AssumeRoleWithWebIden`	Entra ID Workload Identity Federation (*Federated Creds*)
Observabilidade e OTLP	Cloud Logging, Monitoring, Trace, Error Reporting e API OTLP	CloudWatch (Logs, Metrics, Synthetics, Alarms) + X-Ray / ADOT	Azure Monitor + Log Analytics + Application Insights
Eixo 2 — Experiências (Bundles Estáticos, CDN, Assets e Planos)			
Bundles imutáveis (`-exp`)	Cloud Storage + Backend Bucket no LB +	Amazon S3 + CloudFront (*Origin*	Azure Blob Storage + Azure Front Door CDN

3

Eixo 1 — Plataforma (Borda, Hub, Banco, Mensageria e Operação)

O eixo **Plataforma** compreende todas as fundações compartilhadas que mantêm um contrato B2G no ar: a borda pública com proteção WAF e gestão de certificados TLS, a hospedagem do `apps/hub` (Next.js App Router), o banco de dados relacional com isolamento por escola (*Row-Level Security*), a rede privada, o barramento assíncrono de telemetria, a gestão de identidades sem chaves estáticas e a pilha de observabilidade.

Ficha do Eixo 1 — Fundação da Plataforma	
Arquivos Terraform	`loadbalancer.tf`, `cloudarmor.tf`, `cloudrun.tf`, `database.tf`, `network.tf`, `bastion.tf`, `pubsub.tf`, `iam_github.tf`, `secrets.tf`, `registry.tf` e `observabilidade.tf`.
Aderência por nuvem	GCP (ATUAL) NATIVO / DIRETO · AWS EQUIVALENTE · AZURE NATIVO / DIRETO
Ponto focal	No GCP e no Azure (via Front Door), um único recurso de borda unifica o roteamento por *hostname* para contêineres e buckets estáticos sob a mesma política de WAF. Na AWS, o ALB não roteia para S3, o que exige colocar o **Amazon CloudFront** como camada de entrada única diante tanto do S3 quanto do ALB.

3.1 Borda única, WAF e gestão de certificados TLS

PLT-01 Load Balancer Global, Certificados por DNS e WAF OWASP
GCP (ATUAL) NATIVO / DIRETO · AWS EQUIVALENTE · AZURE NATIVO / DIRETO

Como é hoje (GCP)

Um **Global External Application Load Balancer** detém um único IP anycast estático e um `url_map` que distribui o tráfego por *hostname*: `hub_domain` → Cloud Run `frontend`, `api_domain` → Cloud Run `api`, `exp_domain` → Backend Bucket `-exp` com

Cloud CDN, e `assets_domain` → Backend Bucket `-assets` com Cloud CDN. A proteção de borda usa o **Cloud Armor Standard** anexado aos backends de computação, aplicando regras pré-configuradas **OWASP CRS 3.3** (`sqli-v33-stable`, `xss-v33-stable`, `lfi-v33-stable`, `rfi-v33-stable`, `rce-v33-stable`, `scannerdetection-v33-stable`, `protocolattack-v33-stable`, `sessionfixation-v33-stable`) em nível de sensibilidade 2, além de *rate limit* por IP (`300` a `2000` RPM conforme o ambiente, retornando HTTP 429). Os certificados TLS usam o **Certificate Manager** com validação por DNS (`_acme-challenge`): cada *hostname* tem certificado próprio em um `Certificate Map`, o que permite adicionar novos domínios sem tocar nos certificados já ativos e provisionar o TLS **antes** de virar o apontamento DNS (`loadbalancer.tf`).

Equivalente AWS

Amazon CloudFront + AWS WAF v2 + AWS Certificate Manager (ACM) + Application Load Balancer (ALB). Como o ALB da AWS não aceita um bucket S3 como *Target Group* para servir arquivos estáticos diretamente, a borda unificada precisa ser o CloudFront (ou múltiplos ALBs/distribuições separadas). No CloudFront, associa-se uma WebACL do AWS WAF (*AWSManagedRulesCommonRuleSet*, *SQLiRuleSet*, *KnownBadInputsRuleSet* e regra *Rate-based* por IP). O ACM emite certificados gratuitos validados por CNAME no DNS (`us-east-1` para CloudFront), também sem janela de indisponibilidade. Para impedir que alguém contorne o WAF acessando o ALB diretamente, o ALB exige cabeçalho secreto rotacionado ou *prefix list* gerenciada do CloudFront no Security Group.

Equivalente Azure

Azure Front Door (Standard ou Premium) + Azure WAF Policy + Managed Certificates. O modelo do Front Door é arquiteturalmente idêntico ao do Global External LB do GCP: um ponto global anycast que aceita múltiplos domínios validados por CNAME de DNS (*prevalidated custom domains* sem tempo fora do ar) e roteia, por *hostname*, tanto para origens de contêiner (Azure Container Apps) quanto para origens de armazenamento (Azure Blob Storage), com cache CDN e regras OWASP DRS + *rate limit* na mesma borda.

Critério de borda	GCP (LB + Cloud Armor)	AWS (CloudFront + ALB + WAF)	Azure (Front Door + WAF)
Contêiner + Bucket no mesmo recurso	**Sim** (*Serverless NEG* + *Backend Bucket*)	**Só via CloudFront** (exige ALB adicional para Fargate)	**Sim** (origens ACA e Blob no mesmo perfil)

Fechamento do endereço direto do contêiner	Nativo (`INGRESS_TRAFFIC_INTERNAL_LOAD_BALANCER`)	Exige Security Group restrito à *prefix list* do CloudFront + ALB interno	Nativo no ACA com VNet + *Private Link* no Front Door Premium
Adição de domínio sem recriar cert ativo	**Sim** (`Certificate Map` com 1 cert por host via DNS)	**Sim** (certificados ACM independentes por domínio/SAN DNS)	**Sim** (*Managed Certificate* individual por domínio no Front Door)
Modo *preview* de regras WAF	Nativo (`preview = true` por regra ou na política inteira)	Nativo (ação `Count` em cada regra gerenciada do AWS WAF)	Nativo (modo `Detection` vs `Prevention` na WAF Policy)

3.2 Hospedagem do Hub (`apps/hub`) e Identidade (OIDC)

PLT-02 Hub Next.js (SSR + BFF) e Federação OIDC / JWKS

GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Como é hoje (GCP)

O `apps/hub` (Next.js App Router, saída `standalone`) roda como serviço no **Cloud Run** (`1` a `2` `vCPU`, `1` a `2` `GiB` de RAM, concorrência 80, `min_instances = 1` para eliminar *cold start* no horário de entrada escolar e manter o cache de sessão quente). Ele atua como BFF, autentica o usuário contra o **WSO2 Identity Server** via OIDC padrão, assina em memória os tokens de curta duração (15 min, RS256) das experiências com a chave privada injetada do Secret Manager e publica a chave pública em `/.well-known/jwks.json`. A API busca esse JWKS pelo endereço do Hub (`OIDC_JWKS_URI`).

Equivalente AWS

Amazon ECS on AWS Fargate (ou **AWS App Runner**). Embora existam soluções específicas para Next.js (como AWS Amplify Hosting), o Hub da Unooria funciona como emissor OIDC (`issuer`) com chaves montadas via segredo e comunicação controlada com a API; mantê-lo como contêiner OCI padrão ("output: 'standalone'") no ECS Fargate preserva 100% de paridade com o Dockerfile existente. Uma tarefa Fargate (`1 vCPU / 2 GB`, `desired_count >= 1`) atrás do ALB cumpre exatamente o papel do Cloud Run.

Equivalente Azure

Azure Container Apps (ACA). Recebe a mesma imagem Docker do Hub sem nenhuma alteração, mantendo `minReplicas = 1`, escalonamento automático por concorrência HTTP (via KEDA embutido) e injeção de segredos do Azure Key Vault como variáveis de ambiente.

ATENÇÃO

Armadilha de roteamento entre Hub e API na nuvem. No GCP, quando o Hub tem saída de rede configurada como `PRIVATE_RANGES_ONLY` e tenta chamar a API pelo endereço `*.run.app`, a borda do Google responde 404 (`variables.tf`, linha 240). Por isso o Hub chama a API pelo domínio público do Load Balancer (ou exige `ALL_TRAFFIC` via Cloud NAT). Na **AWS** (ECS Fargate) e no **Azure** (ACA), ambos os serviços vivem dentro da mesma VPC/VNet e podem se comunicar diretamente por DNS interno (*ECS Service Connect* / *Cloud Map* na AWS, ou o FQDN interno `*.internal.<env>.azurecontainerapps.io` no Azure), economizando tráfego de NAT Gateway e evitando que as chamadas BFF contem no *rate limit* por IP do WAF externo.

3.3 Banco de Dados Relacional, RLS, Rede Privada e Acesso Dev

O coração da segurança multi-escola da Unooria mora no PostgreSQL 16: todas as tabelas com dados de estudante ou escola possuem `escola_id NOT NULL` e políticas `FORCE ROW LEVEL SECURITY`, com duas *roles* distintas em execução (`app_runtime` para o tráfego isolado por `SET LOCAL app.escola_id` dentro de `comTenancy`, e `app_auth` restrita à resolução de login antes de saber a escola), nenhuma das duas com atributo `BYPASSRLS` nem `SUPERUSER`. Além disso, a tabela `telemetria_eventos` usa particionamento declarativo mensal.

PLT-03 PostgreSQL 16 Privado, Alta Disponibilidade, Pooler e Bastion

GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Como é hoje (GCP)	**Cloud SQL for PostgreSQL 16** (`db-custom-2-7680` em produção, `availability_type = REGIONAL` com *failover* automático entre zonas, backups diários + *Point-in-Time Recovery* via WAL, sem IPv4 público). A conectividade usa *Private Service Access* (peering VPC) e os serviços Cloud Run chegam ao banco via **Direct VPC Egress**. Para acesso de desenvolvimento e aplicação do script de endurecimento de *roles*, há uma VM **Bastion** (`e2-micro`, incluída no *free tier* em `us-east1`) sem IP público, acessada via túnel TCP do **Identity-Aware Proxy (IAP)** autenticado pelo IAM.
Equivalente AWS	**Amazon RDS for PostgreSQL 16 (Multi-AZ)** em subnets privadas (`db.t4g.large` ou `db.m7g.large`). Para evitar o esgotamento de conexões quando tarefas Fargate escalam, a AWS utiliza o **Amazon RDS Proxy** (cobrado por vCPU do banco). O acesso seguro da máquina do desenvolvedor ao RDS privado é feito de forma idêntica ao IAP por meio de uma instância `t4g.nano` sem IP público usando o **AWS Systems Manager (SSM) Session Manager** (`AWS-StartPortForwardingSessionToRemoteHost`), autenticado via IAM e sem abrir a porta 22 para a internet.

Equivalente Azure **Azure Database for PostgreSQL — Flexible Server** (versão 16, modo *Zone-redundant HA*, integrado à VNet por delegação de subnet ou *Private Endpoint*). Grande vantagem operacional e de custo do Azure: o Flexible Server possui **PgBouncer embutido** na própria instância (porta `6432`), ativado por uma flag no Terraform sem custo adicional de proxy separado. Para o túnel de desenvolvimento, pode-se usar o **Azure Bastion** (SKU *Standard* para túnel nativo via `az network bastion tunnel`, ou uma VM `B1ls` com acesso JIT/Entra ID).

NOTA

Particularidade de *roles* gerenciadas nos três provedores. No Cloud SQL (GCP), todo usuário criado pela Admin API recebe `cloudsqlsuperuser`; no RDS (AWS), recebe `rds_superuser`; no Flexible Server (Azure), recebe `azure_pg_admin`. Nos três provedores, o passo SQL de endurecimento das contas de execução (`NOSUPERUSER NOBYPASSRLS NOCREATEDB NOCREATEROLE`) permanece idêntico e portável, pois opera dentro do motor padrão do PostgreSQL 16.

3.4 Mensageria de Telemetria Analítica (Push Autenticado)

As experiências enviam lotes de eventos pedagógicos para `POST /telemetry/batch`. A API valida o envelope com Zod, enriquece com dados do JWT (`escola_id`, `estudante_id` opaco) e publica no barramento em menos de 15 ms, respondendo `HTTP 202`. O armazenamento em lote no banco particionado acontece em `POST /telemetry/ingest`, que é uma rota da **própria API** protegida por token OIDC da conta de serviço do barramento — evitando criar e monitorar um serviço *worker* separado (**ADR-05**).

PLT-04 Fila de Telemetria com Entrega Push OIDC e Fila Morta (DLQ)
GCP (ATUAL) NATIVO / DIRETO AWS EXIGE ADAPTAÇÃO AZURE EQUIVALENTE

Como é hoje (GCP) **Cloud Pub/Sub**: um tópico principal (`{contrato}-telemetria`), uma assinatura **Push** que faz `POST` direto em `https://{api_domain}/telemetry/ingest` assinando um JWT OIDC da conta de serviço `pubsub-push` (validado pelo `PubsubPushGuard` da API via JWKS do Google), política de *backoff* exponencial (10 s a 600 s) e redirecionamento para o tópico de fila morta (`{contrato}-telemetria-dlq`) após 5 tentativas falhas (`pubsub.tf`).

Equivalente AWS O **Amazon SQS** padrão é puramente baseado em *pull* (não faz *push* HTTP), e o **Amazon SNS** faz *push* HTTP mas não possui fila persistente longa nem assina tokens OIDC dinâmicos

nativamente (e exige um passo de confirmação de assinatura `SubscriptionConfirmation` no endpoint). Para reproduzir o mesmo desenho na AWS sem manter um *poller* no código da API, há dois caminhos:

1. **SQS + EventBridge Pipes + API Destinations:** a API publica no SQS (com DLQ nativa após 5 tentativas); um *EventBridge Pipe* lê da fila e faz o `POST` na rota `/telemetry/ingest` autenticando via *EventBridge Connection* (OAuth/OIDC ou chave simétrica no Secrets Manager).
2. **SQS Consumer embutido na API:** como a API na AWS já roda com no mínimo 1 tarefa Fargate ligada 24/7, adicionar um consumidor SQS em segundo plano dentro do módulo `telemetry/` da API elimina o EventBridge e mantém o tráfego 100% interno na VPC.

Equivalente Azure

Azure Event Grid (modelo *Push* nativo com *Dead-Lettering* em Blob Storage e entrega HTTP autenticada com token **Microsoft Entra ID / OIDC** gerado por *Managed Identity*) ou **Azure Service Bus** (filas mais robustas, porém baseadas em *pull* via SDK ou acionadas via Dapr/ACA). O Event Grid Push com autenticação Entra ID é o espelho funcional exato da assinatura Push OIDC do Pub/Sub.

3.5 Identidade Federada (CI/CD), Segredos e Observabilidade

PLT-05 CI/CD Keyless, Segredos e Observabilidade OTLP

GCP (ATUAL) NATIVO / DIRETO AWS NATIVO / DIRETO AZURE NATIVO / DIRETO

Identidade e Segredos

Nas três nuvens, o padrão de **zero chaves de longa duração** é suportado de primeira classe:

- **GCP:** *Workload Identity Federation* valida o token OIDC de `token.actions.githubusercontent.com` restrito a `assertion.repository`; segredos ficam no **Secret Manager**; imagens no **Artifact Registry**.
- **AWS:** *IAM OIDC Identity Provider* confia no GitHub Actions via condição `token.actions.githubusercontent.com:sub`; segredos no **AWS Secrets Manager** (injetados no bloco `secrets` da Task Definition do ECS); imagens no **Amazon ECR**.
- **Azure:** *Federated Identity Credentials* em uma *User-Assigned Managed Identity* no Entra ID; segredos no **Azure Key Vault**; imagens no **Azure Container Registry (ACR)**.

Observabilidade e OTLP A plataforma instrumenta logs estruturados (`pino`), métricas customizadas, 13 políticas de alerta, SLO de 99,5% e reexporta os *spans* OpenTelemetry do navegador (enviados pelo SDK das experiências para a API) para o coletor da nuvem (`observabilidade.tf`):

- **GCP: Cloud Logging** (com exclusão das sondas no sink `_Default` para economizar franquia), **Cloud Monitoring** (3 *uptime checks*, alertas e Service Monitoring SLO), **Cloud Error Reporting** e ingestão direta na API **OTLP** oficial (`telemetry.googleapis.com`).
- **AWS: CloudWatch Logs**, **CloudWatch Metrics & Alarms**, **CloudWatch Synthetics (Canaries)** para sondas de saúde (atenção: Canaries na AWS têm custo por execução significativamente maior que os *uptime checks* do GCP) e **AWS X-Ray** (recebendo OTLP via *sidecar* ADOT — AWS Distro for OpenTelemetry — na tarefa Fargate).
- **Azure: Azure Monitor + Log Analytics Workspace + Application Insights**. O Application Insights aceita OpenTelemetry nativamente, agrupa exceções equivalentes ao Error Reporting e oferece *Standard Availability Tests* equivalentes aos *uptime checks* a custo muito baixo.

4

Eixo 2 — Experiências (Bundles Estáticos, CDN e Armazenamento)

Na Plataforma Unooria, uma regra arquitetural inviolável determina que **experiências pedagógicas (`apps/experiencias/*`) nunca viram serviços de contêiner no Cloud Run**. Elas são aplicações cliente puras (Vite + React + `@edu/sdk`), compiladas para arquivos estáticos e publicadas em armazenamento de objetos atrás de CDN sob caminhos versionados e imutáveis: `/{expId}/{versao}/`. Toda inteligência, correção de nota, estado de sessão e chamada de IA acontece no servidor (`apps/api`).

Além do bucket dos *bundles* das experiências (`-exp`), este eixo abrange os três outros buckets de objetos que sustentam o funcionamento das missões e das ferramentas do professor (`storage.tf`): `-assets` (mídia pesada pública e imagens geradas por IA), `-planos` (planos de aula em Markdown e áudios DUA dos professores, privados e versionados) e `-exp-mapas` (*sourcemaps* privados para desofuscação de pilhas de erro).

Ficha do Eixo 2 — Experiências e Armazenamento de Objetos	
Arquivos de referência	`storage.tf`, `loadbalancer.tf`, `scripts/publicar-manifesto.mjs` e `apps/experiencias/AGENTS.md`.
Aderência por nuvem	GCP (ATUAL) NATIVO / DIRETO · AWS NATIVO / DIRETO · AZURE NATIVO / DIRETO
Ponto focal	Este é o eixo de **maior paridade e menor atrito de migração** entre as três nuvens. GCS + Cloud CDN, Amazon S3 + CloudFront e Azure Blob Storage + Azure Front Door oferecem garantias de durabilidade (11 noves), versionamento de objetos, políticas de ciclo de vida e controle de cache por cabeçalho `Cache-Control` virtualmente idênticas.

4.1 Bundles estáticos imutáveis, CDN e isolamento de origem (`-exp`)

O contrato entre o Hub e a experiência embutida no `<iframe>` depende de duas propriedades da infraestrutura de entrega estática:

1. **Imutabilidade e *rollback* instantâneo:** publicar uma experiência copia os arquivos com *hash* para `/{expId}/{versao}/assets/*` com `Cache-Control: public, max-age=31536000, immutable`, e o `index.html` + `experiencia.json` com `Cache-Control: public, max-age=60`. Ativar ou reverter uma versão não apaga nem move arquivos no bucket — apenas aponta o manifesto para a versão desejada (`publicar-manifesto.mjs`), fazendo o *rollback* propagar em até 60 segundos sem invalidar o cache global da CDN.
2. **Origem dedicada para o `postMessage`:** o Hub envia o token JWT para o `<iframe>` via `postMessage` com `targetOrigin` explícito (nunca `*`). Se as experiências fossem servidas por `https://storage.googleapis.com/{bucket}` (ou `s3.amazonaws.com`), elas dividiriam a mesma origem web (*origin*) com qualquer outro bucket público daquela nuvem. Por isso o Load Balancer/CDN expõe o bucket sob domínio próprio do contrato (`exp_domain`, ex.: `https://exp-entreredes.unooria.com`).

EXP-01 Entrega de Bundles Estáticos e Manifesto por CDN ({contrato}-exp)
GCP (ATUAL) NATIVO / DIRETO AWS NATIVO / DIRETO AZURE NATIVO / DIRETO

Como é hoje (GCP)	Bucket **Cloud Storage** (`{project_id}-exp`) com *Uniform Bucket-Level Access*, conectado ao Global External Load Balancer como `google_compute_backend_bucket` com **Cloud CDN** ativado no modo `USE_ORIGIN_HEADERS` (`loadbalancer.tf`). A publicação no CI usa `gcloud storage rsync` com cabeçalhos diferenciados para `assets/` versus `index.html` e `experiencia.json`.
Equivalente AWS	Bucket **Amazon S3** privado (com *S3 Block Public Access* ligado em 100% do bucket) servido exclusivamente pelo **Amazon CloudFront** usando **Origin Access Control (OAC)**. É um pequeno ganho de segurança sobre o Backend Bucket clássico do GCP: na AWS, o bucket S3 nem precisa ter leitura pública (`allUsers`), pois apenas a distribuição CloudFront assinada via SigV4 consegue ler os objetos. A Política de Cache do CloudFront (`UseOriginCacheControlHeaders`) respeita exatamente o `max-age=60` do `index.html` e o `immutable` dos *chunks* JS/CSS, e o deploy usa `aws s3 sync`.
Equivalente Azure	Conta de armazenamento **Azure Blob Storage** (contêiner `exp`) atrás do **Azure Front Door** com CDN habilitado e respeito aos cabeçalhos `Cache-Control` da origem. No Front Door *Premium*, o contêiner Blob também pode permanecer 100% privado via *Private Link* entre a borda da Microsoft e o Storage Account; no *Standard*, usa acesso de leitura de blobs anônimos restrito ao contêiner, igual ao GCS. O deploy usa `az storage blob sync` ou `azcopy sync`.

4.2 Mídia pesada e imagens geradas por IA (`-assets`)

Para manter o repositório Git enxuto (regra inviolável: nenhum binário > 2 MB no Git), vídeos de abertura das missões (como os clipes MP4 da `exp02em1b`), modelos 3D (`mascote-unooria.glb`), áudios longos de podcasts e todas as imagens `.webp` geradas pela IA no preparo do professor (deduplicadas por SHA-256 da descrição) moram no bucket `{contrato}-assets`.

EXP-02 Bucket Público de Assets e Escrita pela API (`{contrato}-assets`)

GCP (ATUAL) NATIVO / DIRETO | AWS NATIVO / DIRETO | AZURE NATIVO / DIRETO

Como é hoje (GCP)
Bucket **Cloud Storage** (`{project_id}-assets`) exposto sob `assets_domain` no mesmo Load Balancer + Cloud CDN, com política CORS permitindo `GET`/`HEAD` a partir das origens do Hub e das Experiências (necessário para carregamento de texturas WebGL/Three.js no mascote 3D e `<canvas>`). A conta de serviço da API possui `roles/storage.objectAdmin` apenas neste bucket e no de planos, gravando as imagens convertidas em `.webp` via `sharp` com `Cache-Control: public, max-age=31536000, immutable`.

Equivalente AWS
Bucket **Amazon S3** (`{contrato}-assets`) atrás do CloudFront (`assets_domain`) com *Response Headers Policy* para CORS (`Access-Control-Allow-Origin` dinâmico por lista de origens do contrato). A *Task Role* da API no ECS Fargate recebe permissão IAM `s3:PutObject` e `s3:HeadObject` restrita ao ARN do bucket, usando o `@aws-sdk/client-s3` (ou a interface compatível S3 do próprio GCS, se desejado unificar código).

Equivalente Azure
Contêiner `assets` no **Azure Blob Storage** atrás do Azure Front Door, com regras nativas de CORS no serviço de Blob e permissão RBAC *Storage Blob Data Contributor* atribuída à *Managed Identity* do Container App da API.

4.3 Guarda versionada de planos de aula e sourcemaps privados

Os dois últimos buckets do Eixo 2 são estritamente privados (`public_read = false`) e nunca são expostos no Load Balancer nem na CDN:

EXP-03 Buckets Privados: Planos Versionados (`-planos`) e Mapas (`-exp-mapas`)

GCP (ATUAL) NATIVO / DIRETO | AWS NATIVO / DIRETO | AZURE NATIVO / DIRETO

Planos e anexos DUA (`-planos`)
Guarda os planos de aula anuais gerados pela IA e editados pelos professores, além dos áudios e materiais de Desenho Universal para a Aprendizagem (DUA). Possui **Object Versioning** ativo e regra de ciclo de vida que retém versões não atuais por

`planos_historico_dias` (180 dias em produção), garantindo que uma edição acidental de plano nunca perca o texto anterior. Só a API lê e escreve; o navegador nunca acessa este bucket diretamente.

Sourcemaps (`-exp-mapas`) Guarda os arquivos `*.js.map` gerados no build das experiências (`sourcemap: 'hidden'`), permitindo desofuscar um *stack trace* capturado em produção sem expor o código original aos estudantes no bucket público `-exp`.

Tradução AWS e Azure Tanto o **Amazon S3** (*S3 Versioning* + regra `NoncurrentVersionExpiration` de 180 dias) quanto o **Azure Blob Storage** (*Blob Versioning* + política de gerenciamento de ciclo de vida `daysAfterVersionCreationGreaterThan: 180`) implementam esse comportamento de maneira declarativa no Terraform, com paridade funcional de 100%.

Tabela 4.1 – Comparativo técnico dos 4 buckets do Eixo de Experiências nas três nuvens

Recurso / Requisito	GCP (Cloud Storage)	AWS (Amazon S3)	Azure (Blob Storage)
SDK na API (`apps/api`)	`@google-cloud/storage` (ADC nativo)	`@aws-sdk/client-s3` (IAM Task Role)	`@azure/storage-blob` (Managed Identity)
Sincronização no CI	`gcloud storage rsync`	`aws s3 sync`	`azcopy sync` / `az storage blob sync`
Proteção da origem pública	Bucket público lido pelo Cloud CDN	Bucket privado lido via **CloudFront OAC**	Contêiner público ou privado via **Private Link**
Versionamento (`-planos`)	`versioning = true` + `num_newer_versions` / `days_since_noncurrent`	`aws_s3_bucket_version` + `noncurrent_version_ex`	`versioning_enabled = true` + `ManagementPolicy` por idade da versão
Custo base (armazenamento Standard)	≈ US$ 0,020 / GB-mês (`us-east1`)	≈ US$ 0,023 / GB-mês (`us-east-1`)	≈ US$ 0,018 a 0,021 / GB-mês (*Hot tier*)

5

Eixo 3 — API (`apps/api`)

Ficha do Eixo 3 — API (`apps/api`: Backend Único, Jobs, SSE e WebSockets)	
Serviços no código	`apps/api/src/*` (17 módulos de domínio) e job de migração `prisma migrate deploy`.
Arquivos Terraform	`cloudrun.tf`, `iam.tf`, `network.tf` e `variables.tf`.
Dimensionamento atual	2 vCPU, 2 GiB RAM (`sharp` + Prisma Client), `min=1`, `max=5`, concorrência 80, CPU sempre alocada (`cpu_idle = false`).
Aderência por nuvem	GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE (ECS Fargate) / INCOMPATÍVEL (Lambda) AZURE NATIVO / DIRETO (Container Apps)

Por decisão arquitetural deliberada (ADR-01 e regra *"Não criar serviço Cloud Run novo sem necessidade demonstrada"*), a `apps/api` concentra **todos** os papéis de backend da plataforma em um único artefato implantável: API REST síncrona, receptor de *push* da fila de telemetria, proxy e orquestrador de chamadas de IA, processador de geração assíncrona em segundo plano (após responder `HTTP 202`) e servidor de conexões persistentes (SSE e WebSockets bidirecionais de até 60 minutos).

Essa consolidação reduz drasticamente o custo operacional para uma equipe de uma única pessoa, mas impõe quatro requisitos técnicos severos ao serviço de hospedagem de contêineres da nuvem escolhida — requisitos que eliminam imediatamente plataformas de funções efêmeras (*Function-as-a-Service* como AWS Lambda ou Azure Functions).

5.1 Por que a API não cabe em FaaS (Lambda / Functions)

Antes de comparar os serviços de cada provedor, é essencial documentar os quatro comportamentos reais do código de `apps/api/` que determinam o modelo de computação:

1. **CPU sempre alocada fora do ciclo de requisição HTTP (`cpu_idle = false`):** Em dois fluxos críticos — a geração de planos de aula com IA (`apps/api/src/planos/`) e a geração de pacotes de missões inéditas (`apps/api/src/geracao/`) — a rota HTTP grava o regis-

tro inicial com status `gerando`, devolve **`HTTP 202 Accepted`** imediatamente ao navegador e dispara a execução pesada de forma assíncrona no mesmo processo Node.js (`void this.processarEmBackground(...)`). Em ambientes onde a CPU é congelada no instante em que a resposta HTTP é enviada (comportamento padrão do AWS Lambda e do modo *request-based* do Cloud Run), a geração simplesmente para no meio ou leva minutos para avançar poucos milissegundos de CPU.

2. **Memória nativa para processamento de imagem (`sharp`) e Prisma Client (2 GiB RAM):** Quando a IA gera ilustrações de capa ou de etapas pedagógicas, a API recebe os bytes brutos e executa conversão em memória para `.webp` (qualidade 85) via `sharp` (biblioteca C++ *libvips*) antes de gravar no bucket `-assets`, mantendo simultaneamente o motor Rust/N-API do Prisma Client e buffers de áudio PCM. O consumo em pico exige **2 vCPU e 2 GiB de RAM** por réplica.
3. **Streaming SSE de longa duração (até 300 segundos) sem bufferização:** As rotas do Agente de IA do Professor (`POST /agente-professor/conversas/:id/mensagens`) e o endpoint de LLM customizado consumido pelo HeyGen LiveAvatar (`POST /ajuda/llm/chat/completions`) mantêm streams `text/event-stream` abertos por vários minutos. No caso do LiveAvatar, se o primeiro chunk SSE demorar mais de 10 segundos (*Time-To-First-Token*), o provedor de vídeo aborta a sessão — razão pela qual a API emite uma `FALA_DE_ESPERA` em 7 segundos caso o Gemini ainda esteja raciocinando. Se o proxy reverso da nuvem retiver chunks em buffer (comportamento de certos gateways serverless), a fala de espera não chega a tempo e o avatar cai.
4. **WebSockets bidirecionais de até 60 minutos (`timeout = 3600s`):** A API sustenta dois canais WebSocket de longa duração:
   - **Mascote com Voz Ao Vivo (`/ajuda/mascote/ao-vivo`, ADR-158):** o navegador transmite áudio PCM 16 kHz pelo WebSocket até a API, que mantém um segundo WebSocket persistente com o **Vertex Gemini Live** (`BidiGenerateContent`) e devolve áudio PCM 24 kHz em tempo real, com orçamento de até 30 minutos contínuos.
   - **Unoocraft Multiplayer (`apps/api/src/unoocraft/`):** servidor Socket.IO embutido na API para sincronização de estado das salas multijogador da turma, exigindo timeout de conexão de até 3600 segundos e roteamento com afinidade de sessão (*sticky sessions*) quando há mais de uma réplica ativa.

5.2 Peças Arquiteturais do Eixo API

API-01 Computação do Backend (apps/api) — CPU Alocada e Concorrência
GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Configuração GCP
`google_cloud_run_v2_service.api` (`cloudrun.tf`): `cpu = "2"`, `memory = "2Gi"`, `cpu_idle = false`, `min_instance_count = 1`, `max_instance_count = 5`, `max_instance_request_concurrency = 80`, `timeout = "900s"` (expansível até `"3600s"`), `startup_cpu_boost = true`.

Sondas de saúde
`startup_probe` e `liveness_probe` HTTP em `GET /health` (porta 3001).

Como é hoje (GCP)
O Cloud Run v2 combina a simplicidade de um PaaS serverless (deploy de imagem em 15 segundos, revisões imutáveis, tráfego dividido para rollback imediato) com controle fino do ciclo de vida da CPU. Ao definir `cpu_idle = false`, as réplicas ativas mantêm as 2 vCPUs 100% disponíveis tanto durante as 80 requisições simultâneas quanto nos intervalos entre elas, garantindo que as tarefas em segundo plano (`HTTP 202`) terminem em segundos. O faturamento de 1 réplica mínima aquecida 24x7 com CPU sempre alocada fica em torno de US$ 68/mês em `southamerica-east1`, escalando até 5 réplicas apenas durante o pico escolar das 8h da manhã.

Alternativas AWS
Na AWS, existem três candidatos com comportamentos muito distintos para o perfil da `apps/api`:

- **AWS Lambda (INCOMPATÍVEL):** Inviável sem reescrever a arquitetura da API. O limite máximo inegociável de execução é de 15 minutos (quebrando sessões de 30–60 minutos do Mascote Ao Vivo e do Unoocraft), o modelo congela o processo imediatamente após o retorno do `HTTP 202` (travando a geração de planos de aula e missões), e WebSockets exigem o **API Gateway WebSocket API**, que não mantém conexão TCP direta com o processo Node.js (transforma cada frame WS em uma invocação HTTP separada, incompatível com o proxy stateful do Gemini Live e com o Socket.IO em memória).
- **AWS App Runner (EXIGE ADAPTAÇÃO):** Mais próximo da experiência do Cloud Run, mas apresenta limitações severas para a Unooria: o timeout máximo de requisição no ingress Envoy do App Runner historicamente trava em 120 segundos (insuficiente para streams SSE longos e conexões WebSocket de 60 minutos), não possui recurso nativo de *one-off Job* para rodar migrações do banco

(prisma migrate deploy) e exige configurações extras para coexistir atrás de um mesmo ALB.

- **Amazon ECS on AWS Fargate (EQUIVALENTE — Recomendado na AWS):** É a escolha tecnicamente sólida na AWS. Uma *Task Definition* Fargate com 2 vCPU e 4 GB RAM (o Fargate exige no mínimo 4 GB para 2 vCPU) tem CPU sempre alocada por natureza, sustenta centenas de conexões concorrentes no mesmo processo Node.js e integra-se nativamente ao Application Load Balancer (ALB). O ponto de atenção é operacional: o escalonamento horizontal do ECS Fargate baseado em métricas do CloudWatch é significativamente mais lento que o do Cloud Run (leva de 90 a 180 segundos para provisionar e registrar uma nova Task no Target Group durante a rampa das 8h da manhã, contra 5 a 10 segundos no Cloud Run).

Equivalente Azure

O **Azure Container Apps (ACA)** no perfil de carga *Consumption* (ou *Dedicated*) é o equivalente direto (NATIVO / DIRETO) do Cloud Run. Enquanto uma réplica está ativa (minReplicas = 1), a CPU permanece alocada ao contêiner sem congelamento pós-resposta HTTP, permitindo que as rotinas HTTP 202 rodem normalmente. O escalonamento via KEDA responde rapidamente a regras de concorrência HTTP (concurrentRequests = "80"), e as sondas startupProbe e livenessProbe mapeiam 1:1 com a declaração atual.

API-02 Conexões de Longa Duração: SSE, WebSockets (60 min) e Afinidade

GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Rotas críticas

SSE: /agente-professor/*, /ajuda/llm/chat/completions. WebSockets: /ajuda/mascote/ao-vivo (voz PCM bidirecional) e /socket.io/* (Unoocraft).

Requisitos de rede

Desativação de bufferização de resposta no proxy, timeout de inatividade/conexão ≥ 3600s e *session affinity* para Socket.IO.

Como é hoje (GCP)

O Cloud Load Balancing Global em conjunto com o Cloud Run suporta upgrade HTTP/1.1 Connection: Upgrade (WebSockets) e streaming SSE sem bufferização intermediária (desde que o Cloud CDN esteja desabilitado no Backend Service da API, como já configurado em loadbalancer.tf). O timeout do serviço Cloud Run permite até **3600 segundos** (60 minutos), e a afinidade de sessão garante que os pacotes de reconexão do Socket.IO atinjam a mesma instância.

Equivalente AWS

No par **ALB + ECS Fargate**, tanto SSE quanto WebSockets funcionam de maneira nativa e estável, mas exigem três ajustes obrigatórios no

Terraform da AWS (sem os quais as conexões caem aos 60 segundos):

1. Elevar o `idle_timeout` do Application Load Balancer do padrão de `60` segundos para **`3600`** segundos (o ALB suporta até 4000 segundos).
2. Habilitar **Stickiness** (afinidade por cookie gerenciado pelo ALB ou duração de sessão) no *Target Group* da API para que o handshake do Socket.IO e o upgrade WebSocket não caiam em Tasks Fargate diferentes quando `desired_count > 1`.
3. Configurar `deregistration_delay.timeout_seconds` no Target Group para pelo menos `300` segundos, evitando que o desligamento de uma Task corte abruptamente uma sessão ativa de voz com o mascote ou uma aula com o Agente do Professor.

Equivalente Azure

O ingress Envoy do **Azure Container Apps** suporta nativamente WebSockets e streaming SSE, além de oferecer afinidade de sessão embutida (`stickySessions.affinity = "sticky"`). Quando exposto atrás do **Azure Application Gateway** (ou Front Door), ajusta-se o *Request Timeout* para `3600` segundos (no Front Door padrão, onde há limite de 240 segundos de inatividade por pacote, os *heartbeats* WS e comentários SSE `: keepalive` enviados a cada 30 segundos pela API mantêm o túnel aberto).

API-03 Job de Migração de Banco (`prisma migrate deploy`)

GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Configuração GCP

`google_cloud_run_v2_job.migrate` (`cloudrun.tf`): mesma imagem da API, comando `["pnpm", "--filter", "@edu/api", "prisma", "migrate", "deploy"]`, 1 vCPU / 1 GiB RAM, `max_retries = 0`, `timeout = "600s"`.

Como é hoje (GCP)

Para nunca executar `prisma migrate deploy` no `entrypoint` das réplicas web (o que causaria corrida de locks no PostgreSQL quando múltiplas réplicas subissem juntas e atrasaria o `startup_probe`), a fundação separa a migração em um **Cloud Run Job** (`{contrato}-migrate`). O pipeline de CI/CD atualiza a imagem do Job, dispara `gcloud run jobs execute --wait` dentro da VPC e só promove o tráfego da API se o Job terminar com código `0`.

Equivalente AWS

Utiliza-se o **Amazon ECS RunTask** (Standalone Task). Define-se uma `aws_ecs_task_definition` dedicada para migração (1 vCPU, 2 GB RAM, comando sobrescrito para `prisma migrate deploy`, anexada às subnets privadas e ao Security Group do RDS). No deploy,

a GitHub Action invoca `aws ecs run-task` seguido de `aws ecs wait tasks-stopped` e verifica o `exitCode` do contêiner antes de atualizar o serviço principal no ECS.

Equivalente Azure O **Azure Container Apps Jobs** (`azurerm_container_app_job` com `trigger_type = "Manual"`, `replica_timeout_in_seconds = 600` e `replica_retry_limit = 0`) é idêntico ao Cloud Run Jobs em semântica, declaração Terraform e disparo via CLI (`az containerapp job start --wait`).

API-04 Processamento em Background (HTTP 202) vs. Filas Dedicadas

GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Fluxos na API Planos de aula (`POST /planos/gerar`) e pacotes de missões (`POST /geracao/pacotes`).

Mecanismo atual Execução assíncrona no próprio processo com `cpu_idle = false` + recuperação automática de registros órfãos em `gerando` há mais de 10 minutos.

Análise nos 3 provedores Tanto no Cloud Run (`cpu_idle = false`) quanto no ECS Fargate e no Azure Container Apps, o padrão atual — disparar a promessa assíncrona no processo da API após responder `HTTP 202` e consultar o status por polling no Hub — funciona sem necessidade de criar um serviço worker separado. Caso um deploy ou desligamento de réplica interrompa uma geração em andamento, a própria API detecta registros presos no estado `gerando` além do limite de tempo (10 minutos) e permite a retentativa limpa pelo professor. No estágio atual de 10 mil estudantes por contrato, manter tudo no mesmo serviço da API permanece a decisão correta em qualquer uma das três nuvens.

5.3 Síntese Comparativa do Eixo API

Requisito da API	GCP (Cloud Run v2)	AWS (ECS Fargate + ALB)	Azure (Container Apps)
CPU ativa pós-HTTP 202	Sim (cpu_idle = false)	Sim (nativo da Task Fargate)	Sim (réplica ativa mantém CPU)
Dimensionamento unitário	2 vCPU / 2 GiB RAM	2 vCPU / 4 GB RAM (mínimo Fargate p/ 2 vCPU)	2 vCPU / 4 GiB RAM (perfil Consumption)
Concorrência por réplica	Até 80 req/instância (configurável até 1000)	Ilimitada pelo Fargate (governada pelo Node.js e CPU)	Regra KEDA HTTP (concurrentRequests = 80)
Velocidade de scale-out	**5 a 15 segundos** (revisão pré-aquecida no pool Google)	**90 a 180 segundos** (pull da imagem + ENI + health check ALB)	**15 a 35 segundos** (provisionamento de pod no ACA)
Timeout máximo (WS / SSE)	**3.600 s** (60 min) nativo no serviço	**4.000 s** no ALB (idle_timeout ajustado)	**3.600 s+** no ACA / App Gateway (heartbeat p/ Front Door)
Migração (prisma migrate)	Cloud Run Job (--wait)	ECS Standalone Task (run-task + wait)	Container Apps Job (job start --wait)

Tabela 5.2 – Comparativo técnico de execução do backend monolítico (apps/api) nas três nuvens.

6

Eixo 4 — Uso de IA (Modelos, Agente, RAG e Voz Ao Vivo)

Ficha do Eixo 4 — Uso de IA (Modelos, Agente do Professor, RAG e Voz Ao Vivo)	
Serviços no código	`apps/api/src/ai/`, `apps/agente-professor/` (Python + Google ADK), `apps/api/src/agente-professor/`, `apps/api/src/ajuda/`, `apps/api/src/planos/` e `apps/api/src/geracao/`.
Arquivos Terraform	`agente_professor.tf`, `cloudrun.tf`, `iam.tf` e `apis.tf`.
Contratos e ADRs	ADR-105 (quotas no banco), ADR-126/127 (Agent Engine + Vertex AI Search), ADR-132 (LiveAvatar), ADR-158 (Mascote Gemini Live) e ADR-161 (painel de custos de IA).
Aderência por nuvem	GCP (ATUAL) NATIVO / DIRETO — AWS EXIGE ADAPTAÇÃO (alto esforço e custo fixo de RAG) — AZURE EXIGE ADAPTAÇÃO (reescrita do agente + custo fixo de Search)

Se nos eixos de Plataforma, Experiências e API a migração entre GCP, AWS e Azure é essencialmente um exercício de infraestrutura (*Terraform*, rede e IAM) sem tocar no código de aplicação, **o Eixo 4 (Uso de IA) é onde reside a assimetria real entre as três nuvens**.

A Plataforma Unooria não consome inteligência artificial apenas como um "chat genérico de texto". Ela opera quatro subsistemas distintos de IA, cada um explorando capacidades específicas do ecossistema **Google Vertex AI**:

1. **Camada Pluggable da API (`apps/api/src/ai/`):** geração de texto pedagógico e correção estruturada com controle de orçamento de raciocínio (`thinkingBudget`) e até `65.536` tokens de saída (planos de aula anuais), geração de ilustrações nativas na proporção `4:5` e síntese de áudio multi-locutor (`multiSpeakerMarkup`) em uma única chamada.
2. **Agente de IA do Professor (`apps/agente-professor/`):** agente autônomo escrito em Python sobre o **Google Agent Development Kit (ADK)** e hospedado no **Vertex AI Agent Engine** (`ReasoningEngine`), que gerencia sessões conversacionais nativas sem banco de dados auxiliar.

3. **RAG Pedagógico Duplo (Vertex AI Search / Discovery Engine):** duas bases de conhecimento com semânticas opostas — uma base estruturada sem conteúdo (`NO_CONTENT`) para filtro exato + paginação integral das habilidades da BNCC e currículos locais, e uma base não-estruturada (`CONTENT_REQUIRED`) com *Layout Parser* de PDFs que devolve trechos com numeração exata de página inicial e final (`pageSpan: pageStart..pageEnd`).
4. **Ajuda Multimodal em Tempo Real (`apps/api/src/ajuda/`):** Mascote 3D com conversação de voz bidirecional nativa (*speech-to-speech* via WebSocket no **Vertex Gemini Live**) e Avatar em Vídeo (**HeyGen LiveAvatar** em modo FULL conectado ao LLM da API com *Implicit Prompt Caching* de uma base de conhecimento de ~ 65 mil tokens).

6.1 Topologia Comparada da Stack de IA

A Figura 6.1 contrasta a arquitetura atual de IA na GCP com as composições equivalentes na AWS e na Azure.

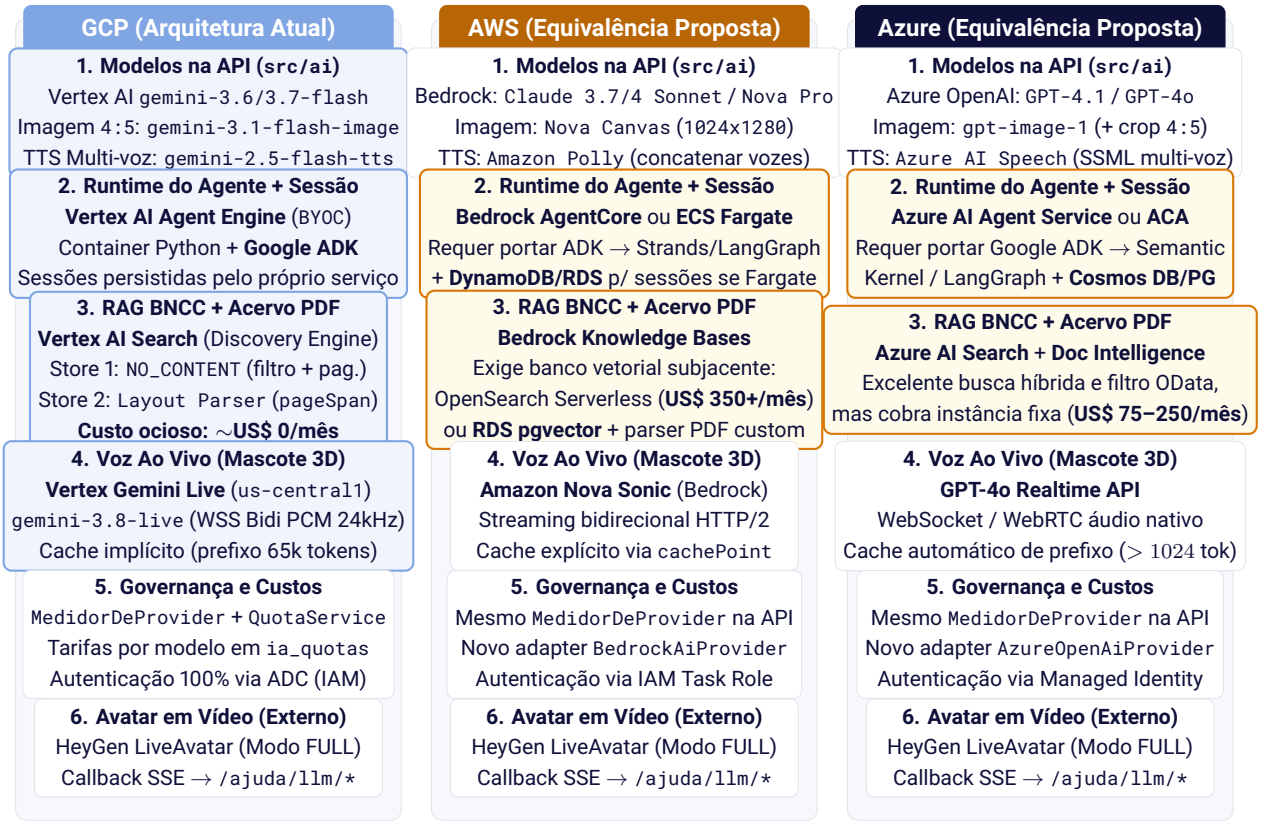


Figura 6.1 – Comparação camada a camada dos subsistemas de Inteligência Artificial entre GCP, AWS e Azure. Os blocos destacados em laranja indicam pontos onde há necessidade de refatoração de código ou degrau relevante de custo fixo mensal.

6.2 Peças Arquiteturais do Eixo Uso de IA

IA-01 Camada Plugável de Modelos na API (`AiProvider`): Texto, Imagem 4:5 e TTS

GCP (ATUAL) NATIVO / DIRETO AWS EXIGE ADAPTAÇÃO AZURE EQUIVALENTE

Código na API	`apps/api/src/ai/ai.provider.ts`, `vertex-gemini.provider.ts`, `medidor.provider.ts` e `quota.service.ts`.
Modelos atuais (GCP)	`gemini-3.6-flash` (padrão), `gemini-3.7-flash` (planos de aula, até 65.536 tokens de saída), `gemini-3.1-flash-image` (imagens `4:5`) e `gemini-2.5-flash-tts` (áudio MP3 com `multiSpeakerMarkup`).
Como é hoje (GCP)	Na API, todo consumo de IA passa pela interface abstrata `AiProvider`, decorada pelo `MedidorDeProvider` (que intercepta o `usageMetadata` de cada chamada e grava o consumo exato de tokens, imagens ou caracteres no PostgreSQL para o painel `/unooria/uso-de-ia`, conforme ADR-161). O `VertexGeminiProvider` explora quatro recursos importantes do Vertex AI:

1. **Orçamento de raciocínio controlável (`thinkingBudget`):** chamadas que exigem resposta rápida (como correções curtas ou interações síncronas de estudantes) enviam `thinkingBudget: 0` (desligando o pensamento oculto para derrubar a latência e o custo de tokens de saída), enquanto a geração de planos de aula e pacotes pedagógicos habilita o raciocínio profundo e janela de saída de até **65.536 tokens** em uma única geração.
2. **Cache implícito de contexto (*Implicit Caching*):** o endpoint da Ajuda (`/ajuda/llm/chat/completions`) injeta a base completa de conhecimento da plataforma (~ 65.000 tokens estáticos) no `systemInstruction`. No Vertex Gemini, o prefixo repetido atinge automaticamente 75% a 90% de desconto (`cached_Tokens`) sem precisar criar ou gerenciar ciclo de vida de objetos de cache.
3. **Proporção nativa `4:5` na geração de imagem:** o `gemini-3.1-flash-image` aceita `imageConfig: { aspectRatio: '4:5' }`, entregando a imagem exatamente na proporção exigida pelos cards e etapas do Design System antes da conversão para `.webp`.
4. **Diálogo multi-locutor nativo em uma única chamada TTS:** na geração de áudios de missões (ex.: diálogos entre personagens ou postagens narradas na Exp08), o **Cloud Text-to-Speech** (`gemini-2.5-flash-tts`) recebe um array de turnos (`multiSpeakerMarkup`) com vozes distintas (ex.: `Aoede` e `Puck`)

e devolve um único arquivo MP3 já mixado e com prosódia conversacional entre os interlocutores.

Equivalente AWS

Como a arquitetura da API já isola o vendor atrás do token de injeção `AI_PROVIDER`, migrar para a AWS exige apenas escrever um `BedrockAiProvider` sem alterar nenhum controller de negócio. Contudo, há diferenças funcionais nos modelos:

- **Texto e JSON Estruturado:** a API *Bedrock Converse* oferece suporte a **Anthropic Claude 3.7 / 4 Sonnet** (com *extended thinking* configurável e saída de até 64k tokens) e à família **Amazon Nova Pro / Lite**. Para aproveitar o desconto de cache nos 65k tokens da Ajuda, o adapter precisará inserir blocos explícitos `cachePoint` no payload do Bedrock.
- **Geração de Imagens (`4:5`):** os modelos **Amazon Nova Canvas** e **Titan Image Generator v2** aceitam resoluções customizadas (ex.: `1024x1280`, que corresponde exatamente à razão `4:5`), permitindo manter a compatibilidade visual.
- **Síntese de Voz Multi-Locutor (EXIGE ADAPTAÇÃO):** o **Amazon Polly não suporta múltiplos locutores em uma única chamada SSML** da mesma forma que o `multiSpeakerMarkup` do Gemini TTS. Para sintetizar um diálogo entre dois personagens no Polly, o `BedrockAiProvider` precisará quebrar o roteiro por turno, disparar N chamadas paralelas ao Polly (uma por fala/voz) e concatenar os buffers de áudio na própria API antes de enviar ao S3.

Equivalente Azure

No ecossistema Microsoft:

- **Texto e Cache:** o **Azure OpenAI Service** (`gpt-4.1`, `gpt-4.1-mini` e série `o3-mini`/`o4-mini`) possui *Prompt Caching* 100% automático para prefixos acima de 1.024 tokens (mesma ergonomia zero-configuração do Vertex Gemini). Limite de saída: `gpt-4.1` suporta até 32.768 tokens de saída (metade dos 65.536 do Gemini 3.7 Flash).
- **Geração de Imagens (`4:5`):** o modelo **gpt-image-1** (e `DALL-E 3`) na Azure trabalha apenas com três resoluções fixas (`1024x1024`, `1024x1536` retrato 2:3 e `1536x1024` paisagem 3:2). O adapter `AzureOpenAiProvider` solicita `1024x1536` e usa a própria biblioteca `sharp` (já presente na API) para aplicar um *center crop* automático para `1024x1280` (`4:5`) junto com a conversão para `.webp`.
- **Síntese de Voz Multi-Locutor (NATIVO / DIRETO):** o **Azure AI Speech** suporta múltiplas tags `<voice name="pt-BR-FranciscaNeural">...</voice><voice name="pt-BR-AntonioNeural">...</voice>` intercaladas

em um único documento SSML e devolve um único stream MP3 pronto.

IA-02 Agente de IA do Professor: Runtime (apps/agente-professor) e Sessões

GCP (ATUAL) NATIVO / DIRETO AWS EXIGE ADAPTAÇÃO AZURE EXIGE ADAPTAÇÃO

Código e stack

apps/agente-professor/agente_professor/ (Python 3.12 + google-adk) + cliente HTTP na API (apps/api/src/agente-professor/agent-engine.client.ts).

Configuração GCP

google_vertex_ai_reasoning_engine.agente_professor (agente_professor.tf): modo BYOC (imagem Docker no Artifact Registry), 4 vCPU, 4 GiB RAM, min_instances = 1, max_instances = 5, concurrency = 36.

Como é hoje (GCP)

O Agente do Professor foi construído em Python utilizando o **Google Agent Development Kit (ADK)** (LlmAgent, ferramentas Python tipadas e callbacks de telemetria) e implantado no **Vertex AI Agent Engine** (ReasoningEngine) em us-central1 (ADR-126). Essa escolha resolve dois problemas em um único recurso Terraform:

1. **Hospedagem gerenciada do contêiner Python (BYOC):** o Agent Engine executa a imagem do Artifact Registry sem que a equipe precise gerenciar cluster Kubernetes ou balanceador interno. A API NestJS chama os endpoints REST nativos do Vertex AI (:query e :streamQuery) autenticada via ADC.
2. **Estado e memória de sessão embutidos (*Zero-DB Session Store*):** o histórico de turnos e eventos da conversa do professor com o agente **não é reenviado pela API a cada mensagem**. O Agent Engine persiste nativamente as sessões (async_create_session, async_get_session, async_list_sessions) e permite renovar o TTL de expiração via PATCH no recurso da sessão (expireTime = +365d a cada nova mensagem). No PostgreSQL da plataforma (agente_professor_conversas), a API guarda apenas o ponteiro sessao_remota_id e o título.

Equivalente AWS

Este é o ponto de **maior acoplamento de código (*lock-in*)** de todo o monorepo:

- **Framework do Agente:** o servidor de produção do Google ADK e seu gerenciador de sessões (VertexAiSessionService) são acoplados ao protocolo do Vertex AI Agent Engine. Na AWS, o caminho natural é migrar a definição do agente (cerca de 400 linhas de Python em apps/agente-professor/agente_professor/) para

o **Strands Agents SDK** (framework oficial open-source da AWS) ou **LangGraph**.

- **Hospedagem e Sessões:** se hospedado no **Amazon Bedrock AgentCore Runtime** (para agentes customizados em contêiner com memória de sessão gerenciada), preserva-se a arquitetura sem banco adicional, mas reescreve-se todo o cliente `agent-engine.client.ts` da API para assinar requisições AWS *SigV4*. Alternativamente, roda-se o contêiner Python como um serviço privado no **ECS Fargate** persistindo os turnos de sessão no próprio **RDS PostgreSQL**.

Equivalente Azure

Na Azure, existem duas abordagens equivalentes:

- **Abordagem 1 — Azure Container Apps interno + PostgreSQL (Mais simples):** empacotar `apps/agente-professor/` como uma aplicação FastAPI + LangGraph/Semantic Kernel rodando em um **Container App com ingress interno** (visível apenas para a API na mesma VNet) e apontando o *checkpointer* de sessão para o próprio **Azure Database for PostgreSQL** já existente no contrato.
- **Abordagem 2 — Azure AI Foundry Agent Service:** serviço gerenciado de agentes da Azure que persiste *Threads*, *Runs* e *Messages* automaticamente (análogo às sessões do Vertex Agent Engine), mas exige remodelar o registro das ferramentas Python.

`IA-03` RAG Pedagógico Duplo: Habilidades (BNCC) e Acervo PDF com Páginas

GCP (ATUAL) NATIVO / DIRETO AWS EXIGE ADAPTAÇÃO AZURE EXIGE ADAPTAÇÃO

Código de busca

`apps/agente-professor/agente_professor/ferramentas.py` e `ingestao/` (`ingerir_habilidades.py`, `ingerir_acervo.py`, `esquema_habilidades.json`).

Configuração GCP

2 Data Stores no **Vertex AI Search** (`google_discovery_engine_data_store` em `agente_professor.tf`): `habilidades` (`NO_CONTENT`, schema estrito) e `acervo` (`CONTENT_REQUIRED`, `layout_parsing_config` + `chunk_size = 500` com `include_ancestor_headings = true`).

Como é hoje (GCP)

O Agente do Professor possui duas ferramentas de consulta com comportamentos que raramente coexistem bem em um banco vetorial simples (ADR-127):

1. **`consultar_habilidades` (Data Store `habilidades` — `NO_CONTENT`):** armazena dezenas de milhares de habilidades da BNCC e dos currículos locais com esquema rígido (`dynamic = "false"`): `codigo` (`EF06MA01`), `anos`

(`["6º ano"]`), `componente_slug`, `etapa`, `unidade_tematica`, `objeto_conhecimento` e `descricao`. Quando o professor pede *"liste todas as habilidades de Matemática do 6º ano"*, a ferramenta aplica filtro booleano exato (`anos: ANY("6º ano") AND componente_slug: ANY("matematica")`) com `query=""` e **pagina até o fim** (blocos de 100 documentos) para devolver 100% da matriz curricular sem omitir nenhuma habilidade. Quando busca por tema, combina o mesmo filtro exato com busca híbrida.

2. **`consultar_acervo` (Data Store `acervo` — `CONTENT_REQUIRED`):** ingere livros didáticos e diretrizes em PDF. O **Layout Parser** nativo do Discovery Engine analisa a estrutura visual do PDF, divide em blocos de 500 tokens preservando os títulos ancestrais (`include_ancestor_headings = true`) e anota em cada chunk o intervalo exato de páginas (`chunkMetadata.pageSpan.pageStart` e `pageEnd`). Assim, o agente cita o título da obra e as páginas exatas.

Além da precisão técnica, o modelo de cobrança do **Vertex AI Search** é **100% serverless por consumo** (poucos dólares por mil consultas e por GB indexado, com **zero custo fixo de cluster ligado**).

Equivalente AWS

O **Amazon Bedrock Knowledge Bases** gerencia a ingestão de PDFs a partir do S3, mas exige um banco vetorial subjacente provisionado:

- **Opção A — Amazon OpenSearch Serverless (Padrão AWS, mas caro):** cada coleção exige um mínimo faturado de **2 a 4 OCUs** funcionando 24x7, gerando um piso fixo de **US$ 175 a US$ 350/mês por contrato** apenas para manter o índice no ar.
- **Opção B — RDS PostgreSQL (`pgvector`) + Full-Text Search (Recomendada na AWS):** aproveita a instância **RDS PostgreSQL 16** já existente no contrato (custo adicional zero de infraestrutura). A ferramenta `consultar_habilidades` torna-se uma query SQL híbrida direta no PostgreSQL (filtros exatos determinísticos + `tsvector` + distância cosseno com embeddings `Titan Text Embeddings v2`), e o `ingerir_acervo.py` extrai os blocos paginados via **Amazon Textract** ou `Docling`/`PyMuPDF` gravando `page_start` e `page_end` na tabela de chunks.

Equivalente Azure

O **Azure AI Search** combinado ao **Azure AI Document Intelligence** (modelo `prebuilt-layout`) entrega a mesma qualidade técnica do Discovery Engine (filtros OData exatos, paginação determinística, busca híbrida com *Semantic Ranker* e chunking por layout com número de página). Contudo, cobra por instância provisionada: o tier

Basic custa cerca de **US$ 75/mês fixos** (e o **Standard S1** custa **US$ 250/mês**). Para eliminar esse custo fixo por contrato na Azure, aplica-se a mesma estratégia de usar **`pgvector`** no Azure Database for PostgreSQL.

`IA-04` Ajuda em Tempo Real: Mascote com Voz Ao Vivo e Avatar em Vídeo
GCP (ATUAL) NATIVO / DIRETO AWS EQUIVALENTE AZURE NATIVO / DIRETO

Código na API `apps/api/src/ajuda/` (`mascote-live.gateway.ts`, `ajuda-llm.controller.ts`, `liveavatar.service.ts`, `ajuda.service.ts`).

Serviços atuais **Vertex Gemini Live** (`gemini-3.8-live` via WSS em `us-central1`) para o Mascote 3D e **HeyGen LiveAvatar** (SaaS externo) + `gemini-3.6-flash` (com cache de 65k tokens) para o Avatar em Vídeo.

Mascote 3D (Voz) No modo Mascote Ao Vivo (`VOZ_AO_VIVO`, ADR-158), a API abre um túnel WebSocket autenticando na Vertex AI (`BidiGenerateContent`) com o modelo **`gemini-3.8-live`** (áudio nativo de entrada PCM 16 kHz e saída PCM 24 kHz + transcrição em tempo real e detecção de interrupção/*barge-in*):

- **Na AWS (EQUIVALENTE):** o equivalente nativo é o **Amazon Nova Sonic** no Bedrock (`amazon.nova-sonic-v1:0`), modelo *speech-to-speech* bidirecional via stream HTTP/2 (`InvokeModelWithBidirectionalStream`), suportando barge-in e transcrição. Exige reescrever o adaptador em `mascote-live.gateway.ts`.
- **Na Azure (NATIVO / DIRETO):** o equivalente direto é a **Azure OpenAI GPT-4o Realtime API** (`gpt-4o-realtime-preview` / `gpt-4o-mini-realtime-preview`), que também opera sobre WebSockets com áudio PCM16 a 24 kHz, detecção automática de atividade de voz (`server_vad`) e eventos JSON muito similares aos do Gemini Live.

Avatar em Vídeo O vídeo do avatar humano (`AVATAR_AO_VIVO`, ADR-132) é processado pelo SaaS externo **HeyGen LiveAvatar** (Modo FULL via WebRTC), sendo **100% agnóstico de nuvem**. Os servidores da HeyGen chamam `POST https://api.<dominio>/ajuda/llm/chat/completions` (SSE); basta que o balanceador da AWS ou Azure não faça bufferização dos chunks SSE para que a `FALA_DE_ESPERA` de 7 segundos funcione nas três nuvens.

IA-05 Governança de Quotas (ia_quotas) e Painel de Custos (ADR-105 / ADR-161)

GCP (ATUAL) NATIVO / DIRETO AWS NATIVO / DIRETO AZURE NATIVO / DIRETO

Código na API	`apps/api/src/ai/quota.service.ts`, `medidor.provider.ts` e `apps/api/src/unooria/uso-ia.service.ts`.
Tabelas PostgreSQL	`ia_quotas` (tetos diários e preços por 1M de tokens configuráveis pelo `admin`), `ia_uso_diario`, `ia_chamadas_log` e `ajuda_limites` (minutos de voz/avatar por escola).
Portabilidade	Por decisão arquitetural (ADR-105 e ADR-161), toda a governança de consumo de IA da Plataforma Unooria foi construída **dentro da própria aplicação e do PostgreSQL**, e não em serviços proprietários de faturamento da GCP. Ao migrar para AWS (Bedrock) ou Azure (Foundry/OpenAI), **o painel de custos de IA e as travas de quota por estudante/professor continuam funcionando sem alterar uma única linha de banco ou de frontend**.

6.3 Síntese Comparativa do Eixo Uso de IA

Capacidade de IA	GCP (Vertex AI)	AWS (Amazon Bedrock)	Azure (AI Foundry / OpenAI)
LLM Rápido + Planos Longos	`gemini-3.6-flash` e `3.7-flash` (saída até **65.536** tok; `thinkingBudget`)	`Nova Pro/Lite` e `Claude 3.7/4 Sonnet` (saída até **64.000** tok)	`GPT-4.1` / `4.1-mini` / `o4-mini` (saída até **32.768** tok no GPT-4.1)
Cache de Contexto (65k tok Ajuda)	**Implícito e automático** (75–90% desc. em prefixos repetidos)	**Explícito** (requer inserir marcador `cachePoint` no prompt)	**Implícito e automático** (50–75% desc. acima de 1.024 tok)
Geração de Imagem `4:5`	`gemini-3.1-flash-image` (`aspectRatio: '4:5'` nativo)	`Amazon Nova Canvas` (resolução `1024x1280` nativa `4:5`)	`gpt-image-1` (`1024x1536` + crop automático `4:5` via `sharp`)
TTS Diálogo Multi-Locutor	`gemini-2.5-flash-tts` (`multiSpeakerMarkup` em 1 chamada)	`Amazon Polly` (exige 1 chamada por fala + concatenação MP3)	`Azure AI Speech` (múltiplas tags `<voice>` em 1 chamada SSML)
Runtime do Agente + Sessão	**Vertex AI Agent Engine** (`BYOC` + Google ADK + sessão nativa sem DB)	**Bedrock AgentCore** ou ECS Fargate (portar ADK → Strands/LangGraph)	**AI Agent Service** ou ACA interno (portar ADK → LangGraph/SK)
RAG Híbrido (BNCC + PDF `pageSpan`)	**Vertex AI Search** (Layout Parser nativo + **US$ 0 fixo/mês**)	**Bedrock KB** (OpenSearch custa **US$ 175–350/mês**; ou `pgvector` no RDS)	**Azure AI Search** + Doc Intelligence (Basic custa **US$ 75/mês**; ou `pgvector`)
Voz Ao Vivo (Mascote 3D)	**Vertex Gemini Live** (`gemini-3.8-live`, WSS PCM 24kHz)	**Amazon Nova Sonic** (stream HTTP/2 bidirecional PCM)	**GPT-4o Realtime API** (WSS / WebRTC PCM 24kHz)

Tabela 6.2 – Comparativo funcional e econômico dos subsistemas de Inteligência Artificial nas três nuvens.

7

Custos (TCO), Operação Solo e Matriz de Migração

Ficha de Síntese Decisória — TCO, Operação Solo e Esforço de Migração	
Unidade econômica	1 contrato público (1 stack dedicada no Terraform) atendendo ∼ 10.000 estudantes (pico de 2.000 a 4.000 simultâneos).
Regiões de referência	GCP `southamerica-east1` (SP) + `us-central1` (IA) | AWS `sa-east-1` (SP) + `us-east-1` (IA) | Azure `brazilsouth` (SP) + `eastus2` (IA).
Recomendação executiva	**Permanecer na GCP como nuvem padrão.** Em editais B2G com exigência contratual estrita de AWS ou Azure, adotar a arquitetura com `pgvector` no PostgreSQL existente para evitar o custo fixo de busca vetorial dedicada.

Em contratos B2G educacionais, o custo de infraestrutura por contrato tem dois componentes muito diferentes: o **piso fixo mensal de prontidão** (aquilo que a nuvem cobra 24x7 apenas para manter a stack isolada do município/estado ligada, com banco HA e réplicas mínimas aquecidas, mesmo durante férias escolares ou domingos) e o **custo variável de tráfego e IA** (proporcional ao uso efetivo pelos estudantes e professores).

Como cada contrato público ganha uma stack fisicamente isolada (um projeto/conta/resource group próprio), qualquer serviço que cobre uma mensalidade fixa mínima elevada multiplica esse custo pelo número de municípios/estados atendidos.

7.1 Estimativa Comparada de TCO Mensal por Contrato (Produção)

A Tabela 7.1 projeta o custo mensal estimado (em dólares americanos, preços públicos sob demanda sem descontos de *Committed Use / Savings Plans*) para **1 contrato em produção** nas regiões de São Paulo (`southamerica-east1`, `sa-east-1` e `brazilsouth`) para computação/dados e EUA para os endpoints de IA generativa.

Tabela 7.1 – Comparativo estimado de TCO mensal (em US$) para 1 contrato B2G de 10.000 estudantes em produção.

Camada / Recurso (Prd)	GCP (US$/mês)	AWS (US$/mês)	Azure (US$/mês)	Justificativa da Diferença
1. Borda, LB Global e WAF	$\sim$ US$ 32	$\sim$ US$ 48	$\sim$ US$ 55	AWS exige CloudFront WAF + ALB regional; Azure Front Door Standard ($ 35 base + WAF policy).
2. Rede Privada e Egress (NAT)	$\sim$ US$ 0–15	$\sim$ US$ 65–110	$\sim$ US$ 45	GCP Direct VPC Egress dispensa NAT para serviços Google; AWS NAT Gateway cobra por hora por AZ em SP.
3. Hub Next.js (`min=1` a `5`)	$\sim$ US$ 38	$\sim$ US$ 52	$\sim$ US$ 42	1 vCPU / 1–2 GiB sempre aquecido + picos diários das 7h30 às 17h30 em dias letivos.
4. API NestJS (`2 vCPU`, `min=1` a `5`)	$\sim$ US$ 95	$\sim$ US$ 135	$\sim$ US$ 110	CPU 100% alocada (`cpu_idle=false`); Fargate exige mínimo de 4 GB RAM para 2 vCPU em `sa-east-1`.
5. PostgreSQL 16 HA (2 vCPU, 8 GiB)	$\sim$ US$ 290	$\sim$ US$ 310	$\sim$ US$ 285	Instância dedicada com failover automático entre zonas (Regional HA / Multi-AZ) + 50 GB SSD + backups 30d.
6. Storage + CDN (4 buckets)	$\sim$ US$ 18	$\sim$ US$ 22	$\sim$ US$ 20	Bundles estáticos com cache longo (1 ano) e mídia otimizada em `.webp`.
7. Fila de Telemetria + Logs	$\sim$ US$ 12	$\sim$ US$ 18	$\sim$ US$ 20	Pub/Sub + Cloud Logging (franquia inicial gratuita de 50 GiB/mês por projeto na GCP).

Camada / Recurso (Prd)	GCP (US$/mês)	AWS (US$/mês)	Azure (US$/mês)	Justificativa da Diferença
8. Runtime Agente Professor (`min=1`)	~ US$ 115	~ US$ 110	~ US$ 95	Contêiner Python aquecido (Agent Engine na GCP vs. ECS Fargate na AWS vs. ACA na Azure).
9. Índice RAG (BNCC + Acervo PDF)	~ **US$** 2	~ **US$** 350 (ou US$ 5 c/ `pgvector`)	~ **US$** 75 (ou US$ 5 c/ `pgvector`)	Vertex AI Search é 100% serverless por query/GB; OpenSearch Serverless tem piso de OCUs; AI Search Basic é fixo.
Subtotal Infraestrutura Fixa/Base	~ **US$** 602	~ **US$** 1.110 (**US$ 765** c/ `pgvector`)	~ **US$** 747 (**US$ 677** c/ `pgvector`)	Na GCP, o piso mensal por contrato é entre **20% e 45% menor** que nas concorrentes.
10. Consumo Variável de IA (10k alunos)	~ US$ 180–320	~ US$ 210–390	~ US$ 220–410	Governado pelo teto em `ia_quotas` e `ajuda_limites`; Gemini Flash e cache implícito reduzem custo na GCP.
TCO Total Estimado / Contrato	**US$ 780 – 920**	**US$ 975 – 1.500**	**US$ 895 – 1.155**	**Vantagem estrutural da GCP em multi-stack B2G**

7.1.1 Três lições econômicas da tabela de TCO

1. **O efeito multiplicador em ambientes de "Desenvolvimento" e "Homologação":** No ambiente `dev` da GCP (`envs/dev.tfvars`), o banco usa tier compartilhado (`db-f1-micro`, sem HA), os contêineres permitem `min_instances = 0` ou `1` leve, não há NAT Gateway pago por hora e o Vertex AI Search custa zero parado. Isso mantém o ambiente `dev` da GCP em cerca de **US$ 45 a US$ 70/mês**. Na AWS, apenas ligar uma VPC com NAT Gateway Multi-AZ, um ALB e uma coleção OpenSearch Serverless em um ambiente de homologação já consumiria mais de **US$ 400/mês antes do primeiro login**.
2. **Em AWS ou Azure, `pgvector` no PostgreSQL é obrigatório para proteger a margem:** Se um edital público exigir contratação em nuvem AWS ou Azure, **não se deve utilizar o Amazon OpenSearch Serverless** para o Agente do Professor. Ao armazenar os embeddings e metadados de página das habilidades BNCC e dos PDFs em tabelas `pgvector` dentro do próprio

RDS PostgreSQL / Azure Database for PostgreSQL já pago pelo contrato, economizam-se de US$ 900 a US$ 4.200 por ano por contrato.

3. **Direct VPC Egress vs. NAT Gateways:** Na GCP, como a API e o Hub se comunicam apenas com o Cloud SQL (IP privado na VPC) e com APIs do Google (Vertex AI, Cloud Storage, Pub/Sub, Secret Manager — todas alcançáveis via *Private Google Access* gratuito), o custo de gateway NAT é virtualmente zero. Na AWS, a menos que se provisionem múltiplos *VPC Interface Endpoints (PrivateLink)* — que também cobram por hora e por AZ —, todo o tráfego da API em subnet privada para o Bedrock, S3, SQS e HeyGen passa pelo **NAT Gateway** tarifado por hora e por GB processado.

7.2 Operação por Uma Única Pessoa (Carga Cognitiva)

A premissa organizacional da Plataforma Unooria (*"Prazo: produção em novembro/2026. Uma pessoa desenvolve. Simplicidade $>$ sofisticação"*) torna a complexidade de manutenção tão importante quanto o custo financeiro. A Tabela 7.2 avalia o esforço diário de operação nos três provedores.

Tabela 7.2 – Comparativo de carga cognitiva e complexidade operacional para um único desenvolvedor.

Dimensão Operacional	GCP (Atual)	AWS	Azure
Quantidade de recursos no Terraform por contrato	~ 45 **recursos** (1 LB global, 2 Cloud Run services, 1 Job, 1 SQL, 4 buckets, 1 Pub/Sub, Agent Engine)	~ 85 **a** 100 **recursos** (VPC, 4+ Subnets, Route Tables, IGW, NAT GW, Security Groups, CloudFront, ALB, Target Groups, ECS Cluster, Task Defs, EventBridge Pipe)	~ 55 **a** 65 **recursos** (VNet, Subnets, Front Door, ACA Environment, 2 Container Apps, 1 ACA Job, Flexible Server, 4 Containers Blob, Event Grid)
Túnel de manutenção ao banco de dados	**1 comando CLI** (`gcloud compute ssh --tunnel-through-iap`) sem gerenciar chaves SSH	**1 comando CLI** (`aws ssm start-session` c/ port forwarding) sem chaves SSH	Exige subir VM de salto + **Azure Bastion** ou abrir regra temporária de firewall
Acesso da API às ferramentas de IA e Storage	**1 único mecanismo IAM** (ADC via Service Account da revisão Cloud Run para SQL, GCS, Pub/Sub e Vertex AI)	**IAM Task Role** unificada (SigV4 para S3, SQS e Bedrock), mas exige conciliar Security Groups de rede + políticas de bucket	**Managed Identity** unificada (Entra ID RBAC para Blob, Event Grid e Azure OpenAI)

Dimensão Operacional	GCP (Atual)	AWS	Azure
Deploy e Rollback de código backend	Revisões imutáveis nativas no Cloud Run (rollback de tráfego em **5 segundos**)	Nova revisão de Task Definition + rolling update no ECS (**2 a 4 minutos**)	Revisões imutáveis nativas no Container Apps (rollback em **10 segundos**)

7.3 Matriz de Portabilidade e Esforço de Migração (Lock-in)

Para responder com precisão à pergunta *"quanto custaria, em dias de engenharia, portar a Plataforma Unooria hoje para a AWS ou para a Azure?"*, a Tabela 7.3 classifica cada parte do sistema pelo seu grau real de acoplamento à GCP.

Tabela 7.3 – Matriz de acoplamento (lock-in) e esforço estimado de engenharia para migrar a plataforma para AWS ou Azure.

Componente / Repo	Acoplamento	O que precisa ser alterado para rodar em AWS ou Azure	Esforço
Hub Web (`apps/hub`, Next.js, OIDC/JWKS)	**Zero** (100% portável)	Nenhuma linha de código. O Dockerfile standalone, a assinatura RS256 dos tokens de experiência e o cliente OIDC funcionam identicamente em ECS Fargate ou Azure Container Apps.	**0 dias**
Experiências e Pacotes (`apps/experiencias/*`, `sdk`, `ui`)	**Zero** (100% portável)	Nenhuma linha de código nas experiências. Apenas ajustar os scripts `scripts/publicar-experiencia.mjs` e `publicar-manifesto.mjs` para suportar `aws s3 sync` ou `az storage blob sync` além de `gcloud storage`.	**0,5 dia**
Banco, RLS e Prisma (`apps/api/prisma`)	**Zero** (100% portável)	Nenhuma linha de SQL ou schema. O PostgreSQL 16 puro com Row-Level Security (`app.current_escola_id`) e `comTenancy` roda sem alterações no Amazon RDS ou no Azure Database for PostgreSQL.	**0 dias**
Storage e Telemetria (`storage`, `telemetry`)	**Baixo** (Isolado em serviços)	Substituir o SDK `@google-cloud/storage` pelo `@aws-sdk/client-s3` (ou `@azure/storage-blob`) nos módulos de planos/geração, e adaptar o publicador de telemetria (`@google-cloud/pubsub` → SQS ou Event Grid). O endpoint receptor `POST /telemetry/worker` permanece igual.	**1,5 a 2 dias**

Componente / Repo	Acoplamento	O que precisa ser alterado para rodar em AWS ou Azure	Esforço
Camada de Modelos IA (`apps/api/src/ai/`)	**Baixo a Médio** (Plugável via DI)	Criar `BedrockAiProvider` ou `AzureOpenAiProvider` implementando a interface existente `AiProvider`. O `MedidorDeProvider` e o `QuotaService` não mudam. Na AWS, adicionar concatenação de turnos para o Polly TTS; na Azure, adicionar crop `4:5` no `sharp` para imagens.	**2 a 3 dias**
Mascote Voz Ao Vivo (`mascote-live.gateway.ts`)	**Médio** (Protocolo WSS)	Adaptar o proxy WebSocket em `apps/api/src/ajuda/mascote-live.gate` para traduzir os pacotes de áudio PCM 16/24 kHz para a **GPT-4o Realtime API** (muito similar na Azure) ou para o stream bidirecional do **Amazon Nova Sonic** (na AWS).	**2 a 3 dias**
Infra como Código (Repo `Fundacao`)	**Alto** (Específico da nuvem)	Escrever um novo módulo raiz de Terraform para AWS ou Azure espelhando as variáveis de `variables.tf` e os ambientes `dev`/`prd`, mais atualização dos workflows de deploy no GitHub Actions (OIDC federado).	**5 a 7 dias**
Agente do Professor (`apps/agente-professor`)	**Alto** (ADK + Agent Engine + Search)	Portar a definição do agente do Google ADK para LangGraph / Strands / Semantic Kernel, reescrever `agent-engine.client.ts` na API e migrar a ingestão/consulta das habilidades BNCC e dos PDFs paginados para **Azure AI Search** ou **PostgreSQL `pgvector`**.	**5 a 8 dias**
Total para Migração Completa	**Arquitetura desacoplada**	85% do código do monorepo (Hub, Experiências, SDK, Design System, Regras de Negócio e Banco RLS) possui **zero lock-in**. Todo o esforço concentra-se no Terraform, nos adapters de IA/Storage e no Agente do Professor.	**16 a 23 dias úteis**

7.4 Conclusão e Diretriz Arquitetural

A análise detalhada dos quatro eixos confirma que **a escolha da Google Cloud Platform (GCP) para a Plataforma Unooria é tecnicamente e economicamente superior para o contexto atual do negócio**:

1. **Aderência nativa aos 4 eixos com menor complexidade operacional:** o par **Cloud Run v2** (`cpu_idle = false`, `timeout = 3600s`, scale-out em segundos) + **Cloud Run Jobs** resolve tanto o Hub quanto o backend monolítico da API sem a sobrecarga de rede e lentidão de escalonamento do ECS Fargate na AWS.
2. **Vantagem decisiva no Eixo de IA e RAG:** a combinação de **Vertex Gemini 3.6/3.7 Flash** (saída de 64k tokens, controle de `thinkingBudget`, cache implícito de 65k tokens, imagens `4:5` nativas e TTS multi-locutor em chamada única), **Vertex AI Agent Engine** (hospedagem do agente Python com sessão embutida sem banco extra), **Vertex AI Search** (Layout Parser com `pageSpan` e custo fixo zero) e **Vertex Gemini Live** entrega capacidades que, na AWS ou na Azure, exigiriam costurar múltiplos serviços distintos e assumir pisos mensais fixos mais altos.
3. **Portabilidade preservada como seguro estratégico:** graças às decisões de engenharia já tomadas no monorepo — PostgreSQL puro com RLS, autenticação OIDC padrão, experiências estáticas desacopladas e interface `AiProvider` plugável —, cerca de **85% da base de código não sabe em qual nuvem está rodando**. Caso um contrato governamental específico exija implantação em **Azure** (a segunda colocada em aderência operacional, graças ao **Azure Container Apps** e à **GPT-4o Realtime API**) ou em **AWS** (via **ECS Fargate** + **RDS PostgreSQL com `pgvector`**), a migração completa é executável em cerca de 3 a 4 semanas de engenharia sem alterar nenhuma tela, experiência ou regra de negócio.

A

Código-Fonte Mermaid dos Diagramas

Em conformidade com a convenção da documentação técnica da Plataforma Unooria, este apêndice preserva a representação em **Mermaid** equivalente aos diagramas **TikZ** renderizados nos capítulos anteriores, facilitando a manutenção futura por agentes de código e ferramentas externas.

A.1 Topologia Geral Comparada (Figura 2.1)

```
flowchart TB
  subgraph GCP["1. GCP (Arquitetura Atual)"]
    G_EDGE["Cloud DNS + Cloud Armor WAF + Global HTTPS LB"]
    G_CDN["Cloud CDN + 4 Buckets GCS (-exp, -assets, -planos, -mapas)"]
    G_RUN["Cloud Run v2: Hub (Next.js) + API (NestJS 2vCPU/2GiB) + Job (migrate)"]
    G_ASYNC["Cloud Pub/Sub (Push OIDC) -> POST /telemetry/worker"]
    G_DB["Cloud SQL PostgreSQL 16 (HA Regional + RLS + Private IP)"]
    G_AI["Vertex AI: Gemini 3.6/3.7 + Agent Engine (ADK) + Vertex AI Search + Gemini Live"]
    G_EDGE --> G_CDN
    G_EDGE --> G_RUN
    G_RUN --> G_ASYNC
    G_RUN --> G_DB
    G_RUN --> G_AI
  end

  subgraph AWS["2. AWS (Equivalência)"]
    A_EDGE["Route 53 + AWS WAF + CloudFront + ALB"]
    A_CDN["CloudFront OAC + 4 Buckets S3"]
    A_RUN["ECS Fargate: Hub + API (2vCPU/4GB) + Standalone Task (migrate)"]
    A_ASYNC["SQS + EventBridge Pipe (API Destination) -> POST /telemetry/worker"]
    A_DB["RDS PostgreSQL 16 Multi-AZ + RDS Proxy + RLS"]
    A_AI["Amazon Bedrock: Claude/Nova + AgentCore + RDS pgvector + Nova Sonic"]
    A_EDGE --> A_CDN
    A_EDGE --> A_RUN
    A_RUN --> A_ASYNC
    A_RUN --> A_DB
    A_RUN --> A_AI
  end

  subgraph AZURE["3. Azure (Equivalência)"]
    Z_EDGE["Azure DNS + Front Door WAF + Application Gateway"]
    Z_CDN["Front Door CDN + 4 Containers Blob Storage"]
```

```
    Z_RUN["Azure Container Apps (ACA): Hub + API + ACA Job (migrate)"]
    Z_ASYNC["Event Grid / Storage Queue -> POST /telemetry/worker"]
    Z_DB["Azure Database for PostgreSQL 16 Flexible Server (Zone-Redundant + PgBouncer)"]
    Z_AI["Azure AI Foundry: GPT-4.1 + ACA/Agent Service + AI Search/pgvector + GPT-4o Realtime"]
    Z_EDGE --> Z_CDN
    Z_EDGE --> Z_RUN
    Z_RUN --> Z_ASYNC
    Z_RUN --> Z_DB
    Z_RUN --> Z_AI
  end
```

A.2 Topologia Comparada da Stack de IA (Figura 6.1)

```
flowchart LR
  subgraph GCP_IA["GCP - Stack de IA Atual"]
    G1["AiProvider: Vertex Gemini 3.6/3.7 Flash + Flash Image (4:5) + Flash TTS Multi-voz"]
    G2["Agente Professor: Vertex AI Agent Engine (BYOC Python + Google ADK + Sessoes Nativas)"]
    G3["RAG Duplo: Vertex AI Search (NO_CONTENT p/ BNCC + Layout Parser pageSpan p/ PDFs)"]
    G4["Voz Ao Vivo: Vertex Gemini Live (gemini-3.8-live WSS Bidi PCM 24kHz)"]
    G1 --> G2 --> G3 --> G4
  end

  subgraph AWS_IA["AWS - Equivalencia de IA"]
    A1["AiProvider: Bedrock (Claude/Nova) + Nova Canvas (4:5) + Polly (concatenar turnos)"]
    A2["Agente Professor: Bedrock AgentCore ou ECS Fargate (Strands/LangGraph + DynamoDB/RDS)"]
    A3["RAG Duplo: RDS PostgreSQL pgvector (evita custo fixo do OpenSearch Serverless)"]
    A4["Voz Ao Vivo: Amazon Nova Sonic (Bedrock Bidirectional Stream PCM)"]
    A1 --> A2 --> A3 --> A4
  end

  subgraph AZ_IA["Azure - Equivalencia de IA"]
    Z1["AiProvider: Azure OpenAI (GPT-4.1) + gpt-image-1 (crop sharp 4:5) + AI Speech SSML"]
    Z2["Agente Professor: ACA Interno ou AI Agent Service (LangGraph/SK + PostgreSQL)"]
    Z3["RAG Duplo: Azure AI Search + Document Intelligence (Layout) ou PostgreSQL pgvector"]
    Z4["Voz Ao Vivo: Azure OpenAI GPT-4o Realtime API (WSS / WebRTC PCM 24kHz)"]
    Z1 --> Z2 --> Z3 --> Z4
  end
```